A Project Report on

OS Tutor: A Fine-Tuned, Retrieval-Grounded and Simulator-Verified Operating Systems Tutor

Submitted by

Divyakush Punjabi (24BCB0126)

Case Study: Building an LLM for Learning and Teaching Operating Systems

October 2026

Abstract

This report describes OS Tutor, a tutor for Operating Systems built on Qwen3-4B-Instruct-2507 and fine-tuned with QLoRA on 11,760 curated training rows. The model is surrounded by three supporting components. Verified simulators compute the answer to every numerical problem, a retrieval (RAG) layer grounds concept explanations in course notes and cites them, and a fact checker corrects known misconceptions. On a 57-item hand-written problem suite the complete system scores 57/57, compared with 30/57 for the untuned model, and on a 600-item out-of-distribution holdout it scores 98.8%.

The RAG layer is built and connected to the tutor, and runs over a corpus of 105 documents: our professor's 42 course documents (lecture slides, assigned textbook chapters, handouts and lab material) plus openly licensed OS texts (Hailperin's *Operating Systems and Middleware* and 55 Wikipedia articles). On 124 labelled questions the correct passage is in the top five results 79.0% of the time, and retrieval takes 8.3 ms (95th percentile) on the CPU. Answers cite their sources and say so when the course materials do not cover a question. On a 200-question theory test, retrieval did not change accuracy (92.0% with it, 93.0% without).

We then built a second knowledge base from openly licensed text only, with no course material. We gathered a pool of 3,291 documents: two open OS textbooks, 1,176 Wikipedia articles, a Wikibook and 2,019 Stack Exchange questions with their answers. From it we kept 2,970 documents, after measuring every passage for quality, relevance to the syllabus and duplication. This knowledge base retrieves a passage containing the answer for 83.3% of 144 short-answer questions, against 74.3% for the course corpus ($p = 0.024$). With either knowledge base, the fine-tuned model's short-answer accuracy rose from about 71% to about 85% ($p \leq 0.001$ over all 144 questions). The open knowledge base, however, did not lead to more correct answers than the course one: 123 against 124 of 144.

The report follows the order of the work: the datasets, how they were preprocessed, the implementation with code screenshots, the measured results, the problems we ran into, and what we would do differently.

Contents

Note on the Results 5

1 Dataset 6

1.1 Training dataset 6

1.2 Evaluation datasets 7

1.3 Retrieval knowledge base 7

1.4 Open retrieval corpus 8

2 Dataset Preprocessing 10

2.1 The eleven-stage build 10

2.2 Cleaning, deduplication and decontamination 10

2.3 Selection, rendering, ordering and release gates 10

2.4 Preprocessing for retrieval 12

2.5 Analysis and compilation of the open corpus 13

3 Code Implementation 15

3.1 System architecture 15

3.2 Data pipeline 16

3.3 Fine-tuning 17

3.4 Verified solver 18

3.5 RAG pipeline 19

3.6 Gateway integration 21

3.7 Evaluation and tests 22

4 Metrics (Assessment): Results and Discussion 27

4.1 Metrics and why each was chosen 27

4.2 Training loss 28

4.3 Procedural accuracy and ablation 28

4.4 Out-of-distribution holdout 29

4.5 Catastrophic forgetting 29

4.6 Fact checker 30

4.7 RAG: retrieval quality and latency 30

4.8 RAG: end to end on the served model 32

4.9 Discussion 36

4.10 Targets and results 37

5 What Went Wrong 38

6 Alternative Flow (Improvements) 41

Appendix A Code listings 43

Appendix B Where every number comes from 50

Note on the Results

Every number in this report was produced by a script in the project repository; Appendix B lists the file each one comes from. Three points need to be kept in mind when reading them.

- **Who wrote the evaluation sets.** The 164 retrieval questions, the 144 short-answer questions and the 130 new theory MCQ items were written by the project team and have not yet been reviewed by the course instructor. They are marked `unreviewed` in the data files.
- **The faithfulness judge.** Whether a cited sentence is supported by its passage was judged by the untuned base model. We checked a sample of 40 of its verdicts by hand and found it lenient (Section 4.8). The required human check by the instructor is still open.
- **Targets.** Section 4.10 lists the targets set before the measurement next to what was measured. A target that was missed is reported as missed.

Three statements in our earlier documents are corrected here and explained in Section 5. No training artifact contains a "final training loss of 0.2710". The gain on the adjacent computer-science subjects is not statistically significant ($p = 0.20$), so it cannot be called transfer. And once the grader was fixed, the adapter's gain on the problem suite without the solver is not significant either ($p = 0.56$).

1. Dataset

1.1 Training dataset

The training set is **`ostutor-sft` v3.0.0**. It is built by one command (`python -m ostutor.build`, seed 42, about 6 minutes), and two consecutive builds are byte-identical. It has five categories, each serving one behaviour the tutor needs:

Category	Share	Rows	What it is	Where it comes from
Procedural	64%	7,526	Worked OS numericals, every decision written as a rule plus the comparison behind it. 39 question types, 8 families.	Our own simulators (verified), plus clarification items where an input is missing
Replay	16%	1,884	General instructions: the anti-forgetting anchor	Tülu 3 SFT mixture, SmolTalk2 (no-think)
Tutoring	8%	938	Socratic replies to a student's *wrong* attempt	MathDial teacher moves (375); own OS dialogues (563)
Shell	6%	707	Natural language → bash, each command passes `bash -n`	tldr, NL2Bash (MIT), InterCode, NL2SH-ALFA
Conceptual	6%	705	Concept explanations	Self-distilled answers to a 295-question bank in 4 wordings, plus hand-written gold
Total (train)	**100%**	**11,760**	**plus 240 validation and 600 out-of-distribution rows**	

Source: `DATASET_CARD.md`, `reports/manifest.json`. Train SHA-256 `f56a7113…477d`.

Procedural families: CPU scheduling 1,733 (FCFS, SJF, SRTF, RR, priority, preemptive priority, HRRN); formula 1,530 (11 kinds: EAT, multilevel paging, deadlock minimum, fork count, …); deadlock / Banker's 1,070; page replacement 990 (FIFO, LRU, OPT, CLOCK); disk scheduling 990 (FCFS, SSTF, SCAN, LOOK, C-SCAN, C-LOOK); contiguous allocation 536; address translation 382; clarification 295.

Why generated data for procedures. A simulator-generated problem has a correct answer by construction, and the team can produce as many as needed at any size. Scraped worked solutions carry their authors' arithmetic slips, and nothing checks them.

Licensing. Every training row is commercially licensed: MIT 8,991 · ODC-BY-1.0 959 · mixed-permissive 925 · CC-BY-4.0 885. Three sources were excluded on licence grounds. `ostep-homework` has no licence and is used only as an offline test oracle. `ceval/ceval-exam` is CC BY-NC-SA and is used for evaluation only. `AnishJoshi/nl2bash-custom` has no licence.

1.2 Evaluation datasets

Set	Items	Measures	Graded by
Hand-written OS problem suite	57	Procedural correctness on free-worded textbook-style problems, 19 kinds	Simulator-exact grader (`harness.py`)
OOD holdout	600	Problems in framings never used in training; extraction, delegation, correctness	Simulator-exact; parameters compared field by field
General MMLU	400	Catastrophic forgetting: general knowledge	Log-likelihood of the four options
Adjacent CS (MMLU CS subjects)	300	Forgetting or transfer in nearby subjects	Log-likelihood
OS theory MCQ	70	Declarative OS knowledge	Log-likelihood of the four options
OS theory MCQ, expanded	200	The 70 above plus 130 new items on topics they miss (IPC, real-time scheduling, allocators, SSDs, file systems, protection, virtualisation)	Generated letter; used for the RAG experiment (Section 4.8)
Fact-checker probes	10	One question per misconception rule	Manual read of each answer (status `unreviewed`)
Retrieval questions	164	124 questions labelled with the document and an evidence phrase from the answering passage (75 answered by course material, 49 by online texts) and 40 questions the corpus does not cover; split in half into dev and test	Recall@k, MRR, abstention
Short-answer questions	144	Short factual questions over 12 OS topics, each with its accepted answers (for example "kernel mode", "supervisor mode", "ring 0"); half dev, half test. A script checks that no question contains its own answer	An accepted answer, as a whole word, in the retrieved passages (answer recall) or in the model's reply (accuracy)

Every training row was decontaminated against all 4,836 evaluation items before selection (Section 2.2). The 144 short-answer questions were written after training and were not part of that check. They test short factual recall, which the training set does not target.

1.3 Retrieval knowledge base

The knowledge base is the directory `data/rag/corpus/`. Its `manifest.json` records, for every document, the source, the owner, the licence or permission, the date it was added and a SHA-256

checksum. Ingestion refuses the whole corpus if any of this is missing, if a file has changed since it was recorded, or if a file is present without a manifest entry.

Part	Documents	Chunks	Contents	Licence / permission
Course material from our professor	42	827	Lecture slides (introduction, processes, limited direct execution, scheduling, MLFQ, proportional share, multiprocessor scheduling, segmentation, paging, page tables, virtual memory, demand paging, locks, semaphores, condition variables, concurrency bugs, threads, memory allocation), nine assigned OSTEP chapters, handouts (deadlock, paging problems, IPC, readers-writers, dining philosophers) and lab material (Unix commands, shell programming)	Supplied by the project owners for use in this tutor; not for redistribution
Hailperin, *Operating Systems and Middleware*, rev. 1.3.1	8	1,956	The OS chapters: introduction, threads, scheduling, synchronisation and deadlocks, virtual memory, processes and protection, files, security	CC BY-SA 3.0
Wikipedia	55		One article per OS topic, from the operating system and system call to inodes, the FAT, disk scheduling and DMA	CC BY-SA 4.0
Total	**105**	**2,783**		

Source: `data/rag/corpus/manifest.json`; per-document chunk counts from `python -m ostutor.rag.ingest data/rag/corpus`.

The professor's material came as one archive of 54 files. Twelve were left out, each with its reason recorded in the manifest: three byte-identical copies, four diagram images with no text, an Office lock file, the assignment brief, two PDF versions of handouts that were also supplied as Word files, and one legacy Word file that Office could not convert automatically. The online texts were added because the course material leaves gaps, for example file-system implementation and disk scheduling. Only text whose licence allows reuse with attribution was used. OSTEP is free to read but not openly licensed, so only the chapters the professor supplied are included. Each online document is fixed to an exact version: a git commit for the book and a revision number for each Wikipedia article.

1.4 Open retrieval corpus

The course material is narrow in places and cannot be shared outside the course. We therefore built a second knowledge base, `data/rag/open/`, with no course material in it at all. We first

gathered a large pool of openly licensed operating-systems text, then measured every passage in it and kept only what passed (Section 2.5).

Source	Pool	Kept	How it was selected	Licence
Hailperin, *Operating Systems and Middleware*	8	8	The eight OS chapters, as in 1.3	CC BY-SA 3.0
Wienand, *Computer Science from the Bottom Up*	9	9	All nine chapters (Unix and C, architecture, the OS, the process, virtual memory, the toolchain, linking), from the book's DocBook source at a fixed commit	CC BY-SA 3.0
Wikipedia	1,176	1,054	Every article in 24 OS categories (process scheduling, memory management, concurrency control, file systems, interrupts, virtualisation and others) plus a core list, each at a recorded revision	CC BY-SA 4.0
Wikibooks, *Operating System Design*	79	51	Every page of the book	CC BY-SA 4.0
Stack Exchange (Computer Science, Unix & Linux)	2,019	1,848	Questions under 30 OS tags (scheduling, deadlocks, paging, semaphores, fork, inode, signals, …) with a score of at least 3, and up to two answers with a score of at least 3. Votes are the quality signal	CC BY-SA 2.5, 3.0 or 4.0 by post date; authors named per document
Total	**3,291**	**2,970**	**18,515 chunks**	

Source: `reports/rag/open_pool_analysis.md`, `data/rag/open/manifest.json`. Fetched by `tools/fetch_rag_pool.py`.

Several well-known sources were left out on licence grounds. The OSDev wiki mixes licences, LibreTexts sets a licence per page that we could not verify, the xv6 book's licence is unclear, OSTEP is free to read but not openly licensed, and several lecture-note collections carry a non-commercial clause. We also considered the OS questions in MMLU-Pro, but only about 14 of them are genuinely about operating systems.

2. Dataset Preprocessing

2.1 The eleven-stage build

`ostutor/build.py` runs every stage in order. It writes each stage's statistics to `reports/manifest.json` and fails the build if any release gate fails.

#	Stage	What it does	Rows after
1	Ingest	31 source files, line-level provenance (file + row index) on every row	80,732
2	Analyse	Each candidate cleaning rule is tested against the data *before* adoption; some were rejected on evidence	80,732
3	Clean / scrub / repair	Refusals, degenerate repetition, non-Latin replay, bash syntax errors and answer-revealing tutor turns dropped; PII redacted (card numbers Luhn-checked); rows with credentials destroyed	70,170
4	Deduplicate	−656 exact (blake2b) · −1,133 near (MinHash LSH, Jaccard 0.80)	68,381
5	Decontaminate	−643 against 4,836 eval items (distinctive 13-grams; exact parameter fingerprints, which caught 15 reworded leaks)	67,738
6	Score	Category-specific quality scorer	—
7	Select gold	Quality floor → identity dedup (fingerprint or embedding) → k-centre diversity → weighted family waterfill	12,000
8	Render	The model's own chat template, one system prompt per (mode, domain)	12,000
9	Split, then order	11,760 / 240 / 600; rectifier interleaving and spaced replay on the train split	11,760
10	Verify	Every ordering invariant re-checked on the rows actually shipped	—
11	Release gates	19 gates; all must pass	19/19

Rows after stages 4 and 5 are computed from the removal counts in the dataset card.

2.2 Cleaning, deduplication and decontamination

- **Analyse before cleaning.** A rule is adopted only if the data supports it, and every verdict is written to `reports/analysis/`. Two bugs found this way are described in Section 5: a tutoring threshold that silently dropped all 1,508 OS tutoring rows, and a decontamination key that removed 58% of the tutoring rows.
- **Boilerplate is not contamination.** A naive "any shared 13-gram" rule removed 58% of the procedural rows. Training and evaluation problems share template text: a Banker's table header alone appears in 428 rows. An evaluation n-gram therefore counts as contamination only when it is *rare* in training, appearing in at most 5 rows. The 93 boilerplate n-grams are ignored.
- **Generated rows are checked by their parameters, not their words.** A leaked problem is the same problem however it is worded, so simulator rows are matched on an exact parameter fingerprint. That caught 15 reworded leaks that n-grams missed.

2.3 Selection, rendering, ordering and release gates

- **Ratios are proportions, not ceilings.** In v1, the categories that came up short let the others' shares balloon: replay reached 31% against a 12% design. The build now scales the total to

the scarcest category, which is tutoring. Measured mixture drift is ≤ 0.02 percentage points.

- **Rectifier ordering.** No more than 4 consecutive rows come from one category, and no more than 3 from one source across 81 sources (measured: 1). Replay is spaced every 4 rows early in training and every 14 late (FOREVER, arXiv 2601.03938). The order matters, so training uses a sequential sampler.
- **Surprise-ranked replay.** Replay rows are chosen by their per-token NLL under the base model (SuRe, arXiv 2511.22367), not by a hand-written quality heuristic.
- **Release gates (19/19 pass).** The gates cover: no secrets, no `<think>` blocks, no non-commercial rows, schema validity, token p99 ≤ 16,384 (measured 2,753; longest row 11,648, so nothing is truncated), rectifier invariants, mixture drift, distinct-answer ratio, template share, diversity, OOD holdout present, procedural inputs complete, and OS tutoring present.

2.4 Preprocessing for retrieval

Step	Choice	Why
Convert legacy formats	Fourteen `.ppt` files converted to `.pptx` with PowerPoint itself (`tools/convert_office_legacy.ps1`)	The extractor reads the modern formats; Office converts its own files losslessly
Extract with structure	Slides become "Slide N: title"; Word headings stay headings; PDF pages become sections, with large-font lines promoted to headings; the Hailperin LaTeX source is converted with pandoc	A citation must name a place a student can find: a slide, a page, a section
Remove running headers	A PDF line that appears on at least a third of the pages is dropped	The OSTEP chapters print the book name and page header on every page, which was being glued into the middle of passages
Normalise text	Unicode NFKC, which turns the typographic ligature "fi" back into "fi"	Otherwise a student typing "file" never matches "file" in a typeset PDF
Validate the manifest	Source, owner, date, checksum and licence or permission per document; unlisted or changed files refuse the corpus	The brief requires provenance and permission for every document
Split by heading	Each heading starts a section; the chunk carries its document title and the full heading path	The citation then reads, for example, "Synchronization and Deadlocks — Deadlock › The Deadlock Problem"
Window long sections	≤ 180 words per chunk with a 30-word overlap	One idea per chunk; a sentence cut at a boundary is still whole in one of the two chunks
Lexical index	Lower-case alphanumeric tokens, a short stop-word list, BM25 ($k_1 = 1.5$, $b = 0.75$)	Exact OS terms (TLB, inode, Belady) matter
Dense index	all-MiniLM-L6-v2 (Apache-2.0, 22M parameters), normalised embeddings of section + text, loaded from the local cache only	Catches paraphrase; runs on the CPU; no query leaves the machine
Integrity	SHA-256 over every chunk id, title, section and text; embeddings cached under that hash	Every evaluation names the exact corpus it ran on, and a restart does not re-embed

The 105 documents give 2,783 chunks (median 180 words). A full build takes about 46 s on the CPU, and reloading an unchanged corpus takes about 7 s, most of it loading the embedding model.

2.5 Analysis and compilation of the open corpus

The pool was cut into the same chunks the retriever uses (21,740 chunks after dropping 164 stub documents) and every chunk was measured. The rules below were written down before any retrieval or end-to-end measurement (`tools/analyze_rag_pool.py`).

Measurement	Rule	Why
Document length	Documents under 120 words are dropped whole	Wikipedia stubs and one-line answers give a passage with nothing to cite
Letter share	A chunk is dropped if fewer than 60% of its characters are letters	Tables of numbers, hex dumps and command output
Code share	A chunk is dropped if more than half its lines look like code	Long code listings answer no conceptual question and crowd out prose
Relevance to the syllabus	Cosine (MiniLM) to the nearest of 40 OS topics, each described in one sentence after the ACM CS2013 OS knowledge area. The floor is the 5th percentile of the two textbooks' chunks: **0.307**	The textbooks are known to be on topic, so the floor keeps 95% of them and is not a number we picked
Near-duplicates	MinHash over word 5-shingles, Jaccard ≥ 0.7. Of two copies, the one from the higher source is kept (textbook > Wikipedia > Wikibooks > Stack Exchange)	Wikibooks copies Wikipedia text; a duplicate wastes one of the four retrieved slots
Answer coverage	For each source, the dev short-answer questions for which some kept chunk contains an accepted answer	The ceiling for retrieval: a question no chunk answers cannot be retrieved

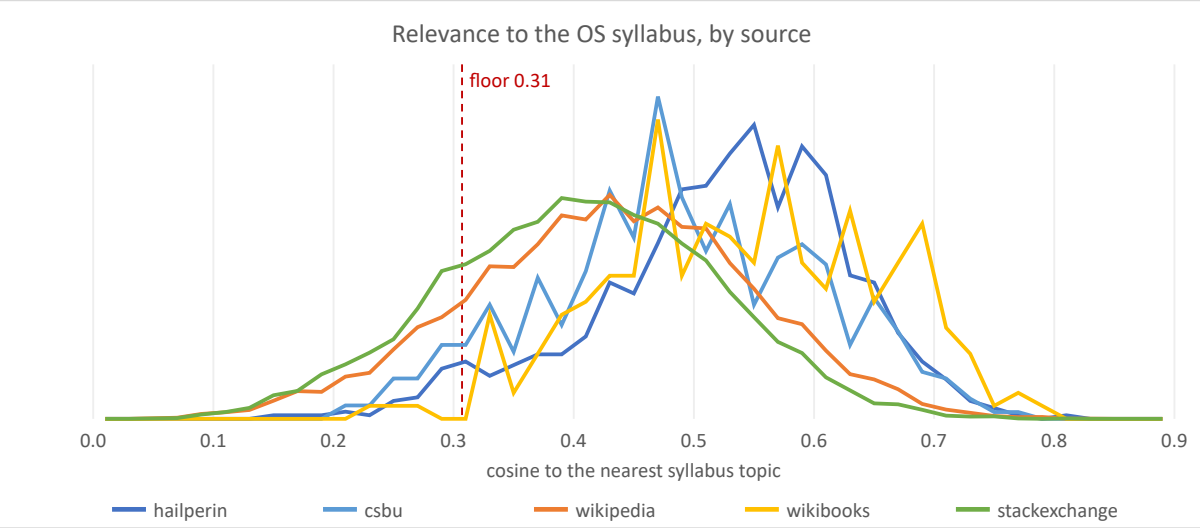


Figure 1: Distribution of chunk relevance by source, with the floor at 0.307. The textbooks and Wikibooks sit to the right; Wikipedia and Stack Exchange have long tails of off-topic text.

Source	Chunks	Kept	Median relevance	Main reasons for dropping
Hailperin	882	800	0.537	mostly code 45, off-syllabus 36
CSBU	471	433	0.484	off-syllabus 27, low letter share 8
Wikipedia	11,395	9,386	0.432	off-syllabus 1,679, mostly code 236, low letter share 89
Wikibooks	243	177	0.544	duplicate of Wikipedia 63
Stack Exchange	8,749	6,624	0.407	off-syllabus 1,602, mostly code 281, low letter share 242
Total	**21,740**	**17,420**		

Source: `reports/rag/open_pool_analysis.md`.

- **Coverage.** The kept chunks contain an answer to 71 of the 72 dev questions; the course corpus covers 65. Wikipedia alone covers 70, and Stack Exchange 63. Hailperin covers 45 and CSBU 30: they are deep but not broad.
- **Topics.** Every one of the 40 syllabus topics has kept chunks. The thinnest are segmentation (14), RAID (39), paging (67) and page replacement (75). The largest are virtual memory and file-system implementation, both with well over a thousand.
- **Compiling.** The kept chunks are regrouped into their sections and written out as documents, each with a manifest entry naming its source, revision, authors and licence. A section is kept whole if any of its chunks passed, so that a citation points at a coherent section. That is why ingestion gives 18,515 chunks rather than 17,420.
- **A control.** The same pool with no filtering (21,715 chunks, `data/rag/open_all/`) was built as well, to measure what the filters are worth (Section 4.7).

The analysis also found a defect in our own conversion. In the first build, 1,137 of the chunks (5.8%) were Wikipedia "See also" lists, "External links" and reference lists. The rule meant to remove those sections compared heading levels off by one. It was fixed and covered by tests, the corpus was rebuilt, and 39 such chunks remain (subsection-level "Notes"). Every number in this report is from the rebuilt corpus.

3. Code Implementation

The code figures below show the source files as they appear in VS Code, with the file name and line numbers visible in each one. The same code is reproduced as text in Appendix A. The terminal screenshots show the output of the commands named in them.

3.1 System architecture

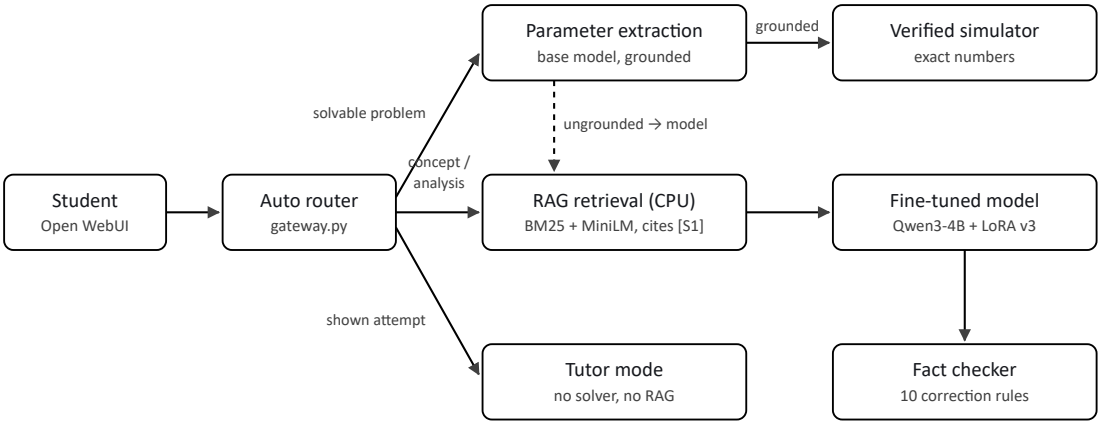


Figure 2: Request flow. The router picks a path. Solvable problems are computed by a simulator. Concept and analysis questions are grounded in retrieved course notes, answered by the fine-tuned model and fact-checked. A shown attempt goes to the Socratic tutor, which never receives a computed answer or a retrieved passage.

Layer	Component	Runs on
Weights	llama.cpp b11050 · Q8_0 base GGUF + v3 LoRA (f16) applied at runtime · port 8081	RTX 4060 Laptop (8 GB)
Logic	`serve/gateway.py` (FastAPI): routing, solver, **retrieval**, fact checking, loop guard, trained prompts · port 8090	CPU
Retrieval	`ostutor/rag/`: corpus loader, BM25, MiniLM, rank fusion, abstain gate, citations	CPU
UI	Open WebUI 0.11.3, local only · port 3000	CPU

3.2 Data pipeline

Stages 3–5 of the build. Each stage writes its statistics into the manifest and prints what it removed, so a regression shows up as a changed count.

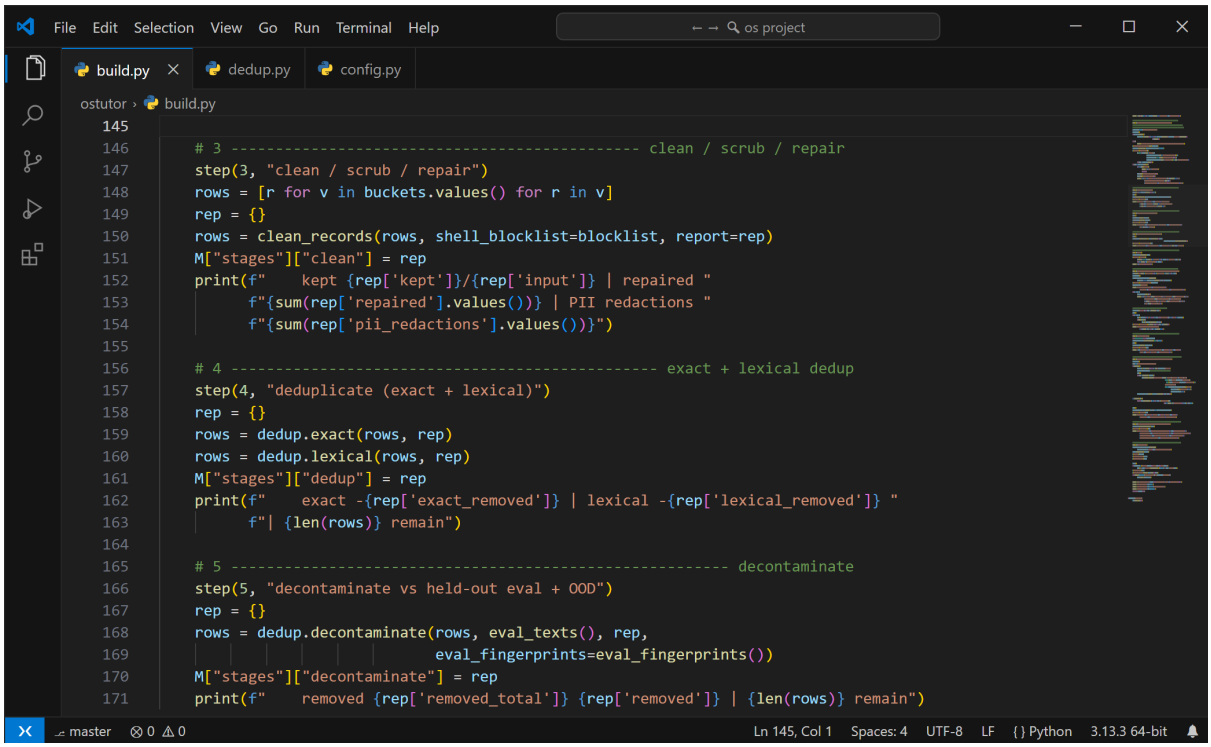


Figure 3: `ostutor/build.py`: cleaning, deduplication and decontamination stages.

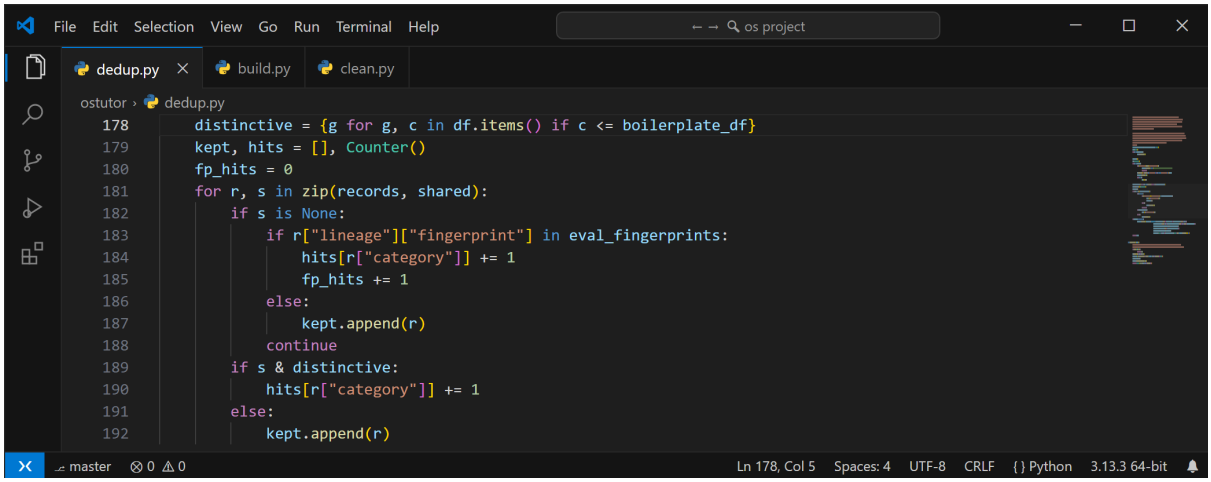


Figure 4: The core of decontamination. Generated rows are judged by exact parameter fingerprint, prose rows by *distinctive* shared 13-grams only.

3.3 Fine-tuning

QLoRA via Unsloth on Kaggle's free tier: 2 × Tesla T4 with DDP (`torchrun --nproc_per_node=2`), 735 optimiser steps, 3.5 hours. The settings and the reason for each:

Setting	Value	Why
Base model	Qwen3-4B-Instruct-2507 (Apache-2.0), non-thinking	Visible numbered steps for students; fits 16k-token QLoRA on a T4; 14B+ cannot train at 16k on free hardware or serve fast on 8 GB
LoRA	r = 32, α = 64, all linear layers, dropout 0.05	LoRA bounds weight drift (forgets less than full fine-tuning)
Learning rate / schedule	1e-4, cosine, 3% warm-up, **1 epoch**	v2.2's second epoch memorised more
Sequence length	16,384, no packing	Longest row 11,648; packing would break the ordering
Precision / optimiser	fp16, adamw_8bit, NEFTune α = 5	T4 (Turing) has no bf16
Loss	Assistant tokens only	Learn to answer, not to echo the question
Checkpoint	Best validation loss, evaluated ~every 8% of steps	The OOD loss is logged but never used to select a checkpoint

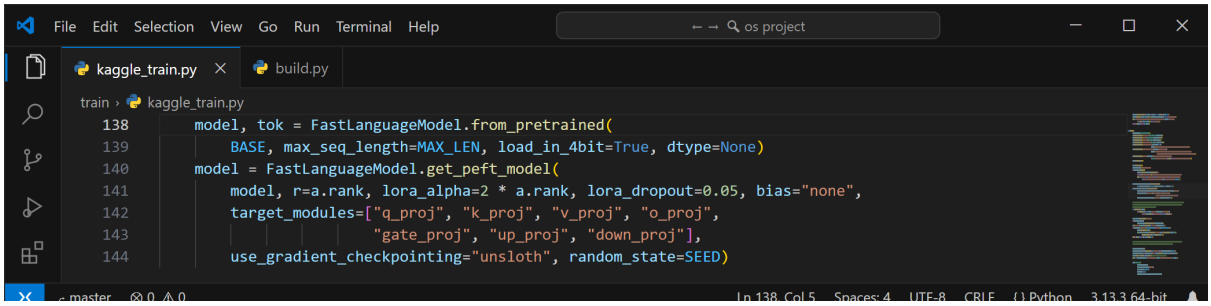


Figure 5: `train/kaggle_train.py`: the 4-bit base and the LoRA adapter on all seven projection matrices.

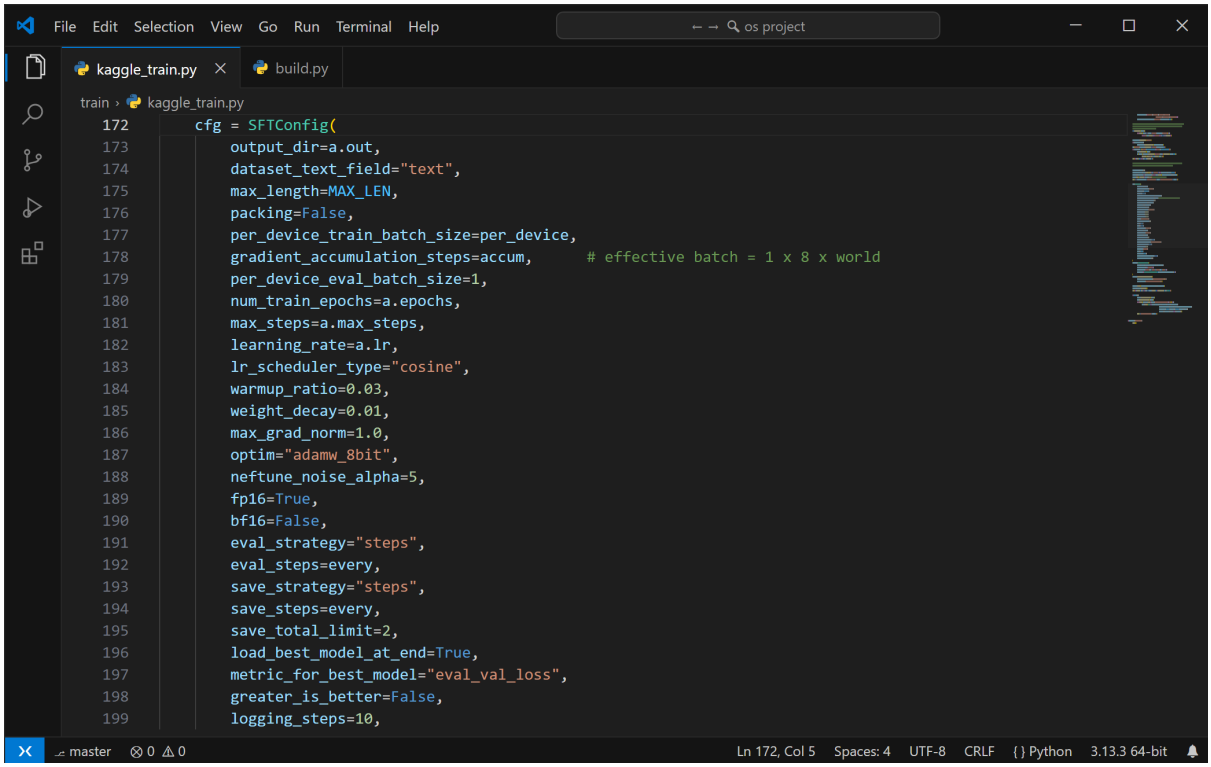


Figure 6: The trainer configuration: no packing, fp16, 8-bit AdamW, NEFTune, step-wise validation with best-checkpoint restore.

3.4 Verified solver

The model never carries out a procedure. It *reads* the question into typed JSON. This step runs on the untuned base at LoRA scale 0, which reads more reliably: 84% extraction accuracy against 79% for the adapter. Every extracted number must then appear in the question. Grounded parameters go to a simulator, which computes the answer; anything ungrounded falls back to the model.

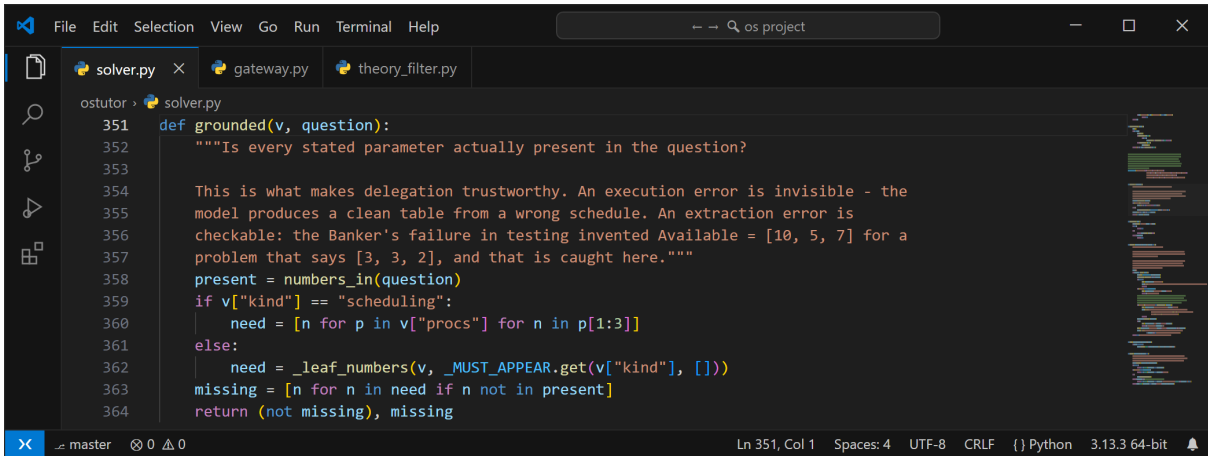


Figure 7: `ostutor/solver.py`: the grounding check, which makes delegation trustworthy. It caught an invented Banker's *Available* vector [10, 5, 7] for a problem stating [3, 3, 2].

3.5 RAG pipeline

The pipeline has four files. `extract.py` turns PDF, PPTX, DOCX, Markdown and text into Markdown whose headings are the citation anchors. `corpus.py` validates the manifest and cuts the documents into citable chunks. `retrieve.py` ranks the chunks, decides whether any of them are relevant, and builds the prompt. `ingest.py` is the command the course staff use to add material (`add`) and rebuild the index.

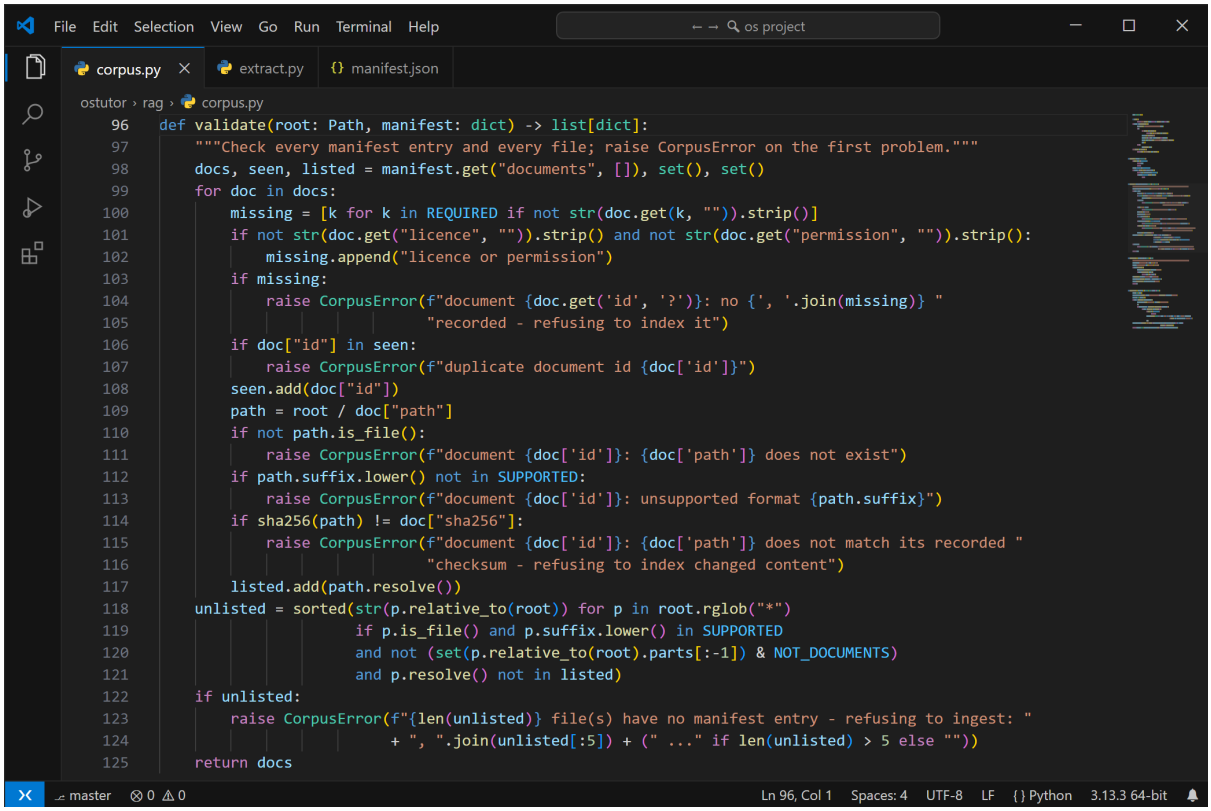


Figure 8: `ostutor/rag/corpus.py`: manifest validation. A missing field, a changed checksum or an unlisted file refuses the whole corpus.

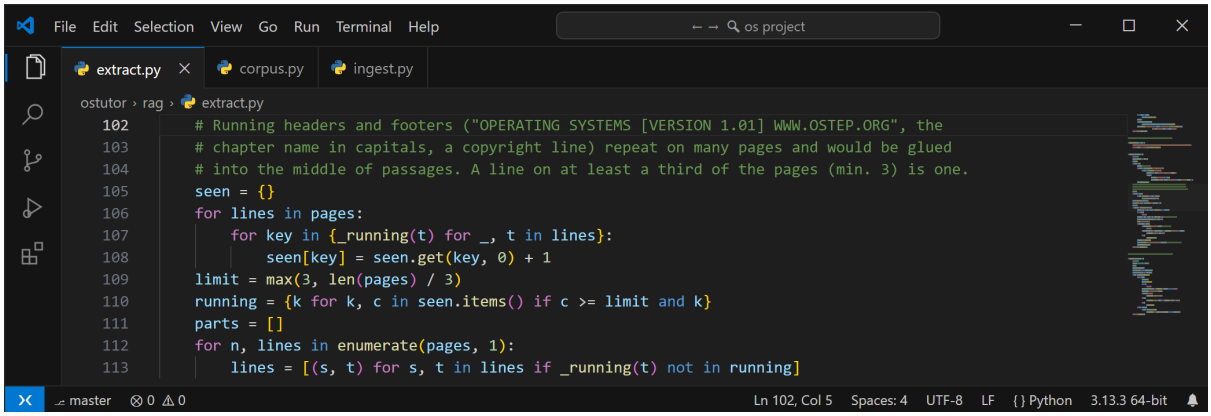


Figure 9: `ostutor/rag/extract.py`: lines repeated on a third of a PDF's pages are running headers and are dropped.

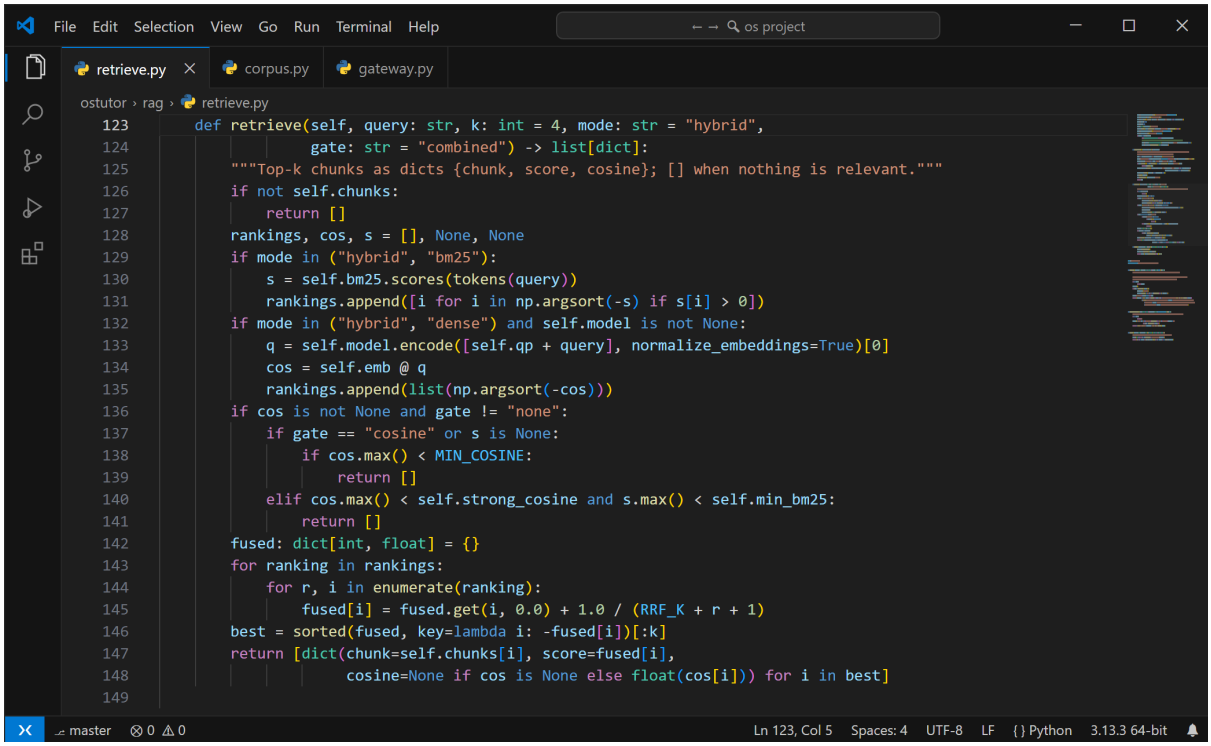


Figure 10: `ostutor/rag/retrieve.py`: BM25 and dense rankings, the abstain gate, and reciprocal-rank fusion (k = 60; Cormack et al., SIGIR 2009).

The abstain gate. A passage that is only the least-bad match is worse than no passage, because the model would cite it as support. The gate keeps the passages if *either* signal is strong: the best dense similarity, or the best BM25 score. BM25 scores grow with the size of the corpus, so the two thresholds are fitted for each corpus. For the course corpus they are cosine ≥ 0.59 or BM25 ≥ 18.5, fitted on the dev half of the questions only (Figure 15).

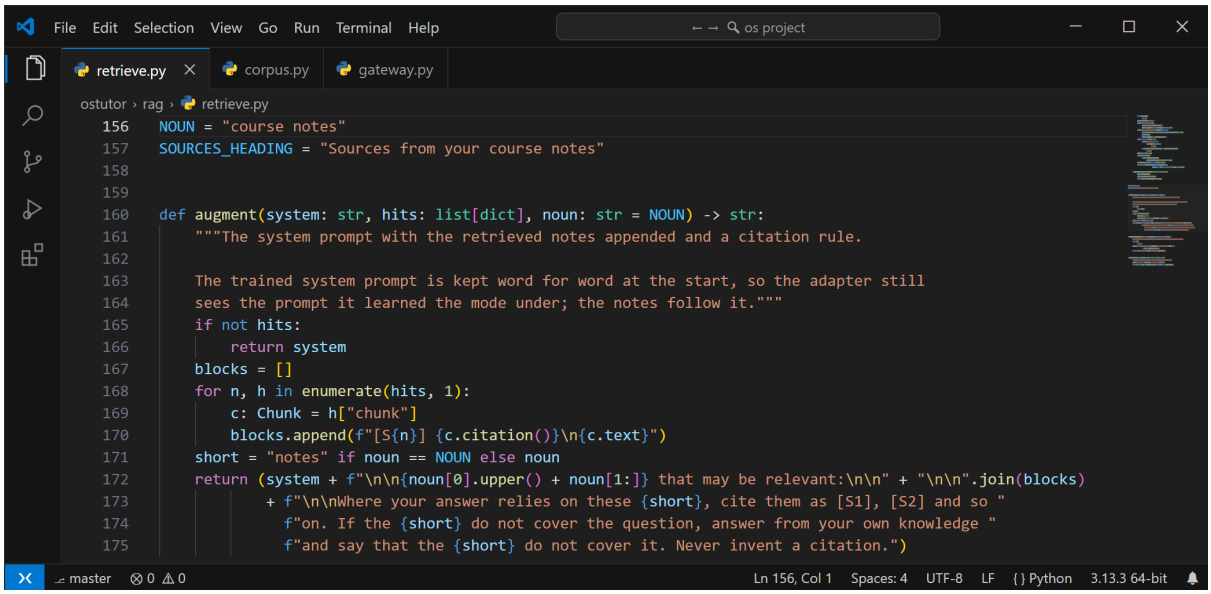


Figure 11: The prompt keeps the *trained* system prompt word for word at the start, so the adapter still sees the prompt it learned the mode under. The notes follow, labelled [S1]…[S4], with an instruction never to invent a citation.

3.6 Gateway integration

Retrieval is layer 2 of the gateway. It runs after the simulator layer, so a solvable problem is still computed and retrieval is not even consulted for it (a gateway test checks this). It runs before the fact checker, which checks the grounded draft. Only `explain` and `analyze` retrieve. Tutor mode never does, because a passage could state the answer the student is working towards. Retrieval is switched on by `OSTUTOR_RAG=data/rag/corpus`; a gateway started without it behaves exactly as before. Answers end with a "Sources from your course notes" list. When the gate abstains, they end instead with "Not found in the course materials", so general knowledge is never passed off as course content.

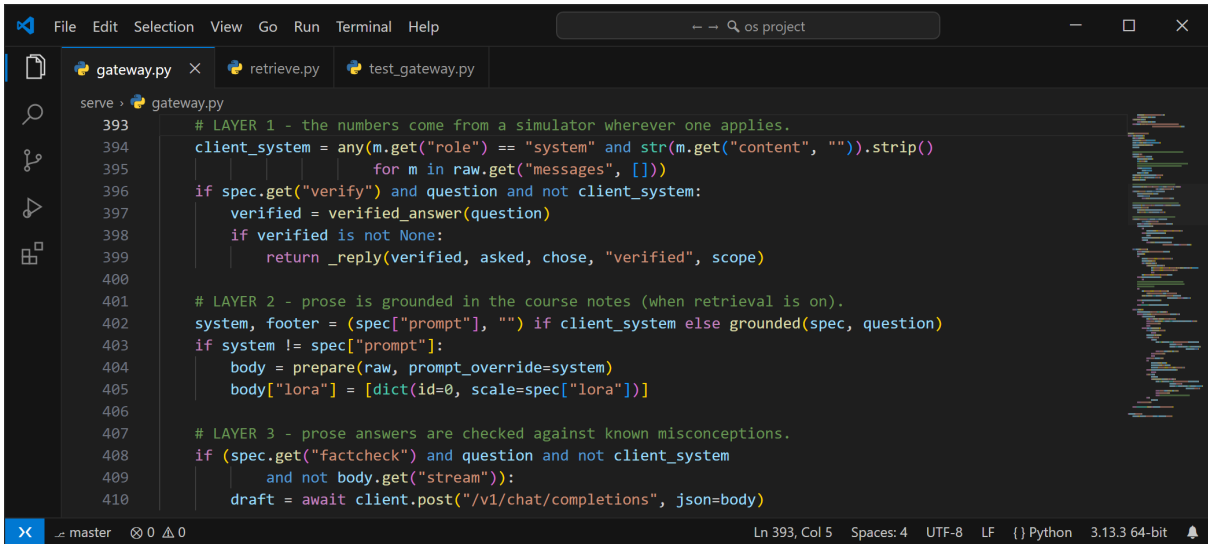


Figure 12: `serve/gateway.py`: layer 1 is the simulator, layer 2 retrieval, layer 3 the fact checker.

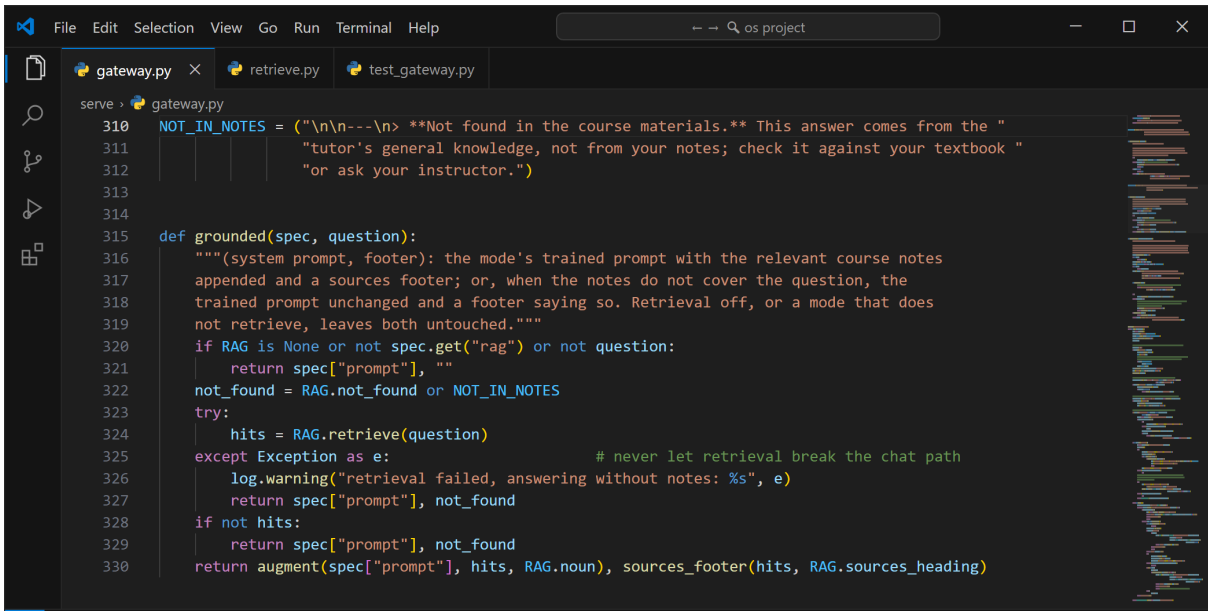


Figure 13: `serve/gateway.py`: the prompt and footer for a question, including the notice when the course materials do not cover it.

3.7 Evaluation and tests

Every comparison in the project is *paired*: the same items are scored under two configurations. The right test for a paired yes/no outcome is McNemar's, and with 5–30 discordant items the exact binomial form is needed.

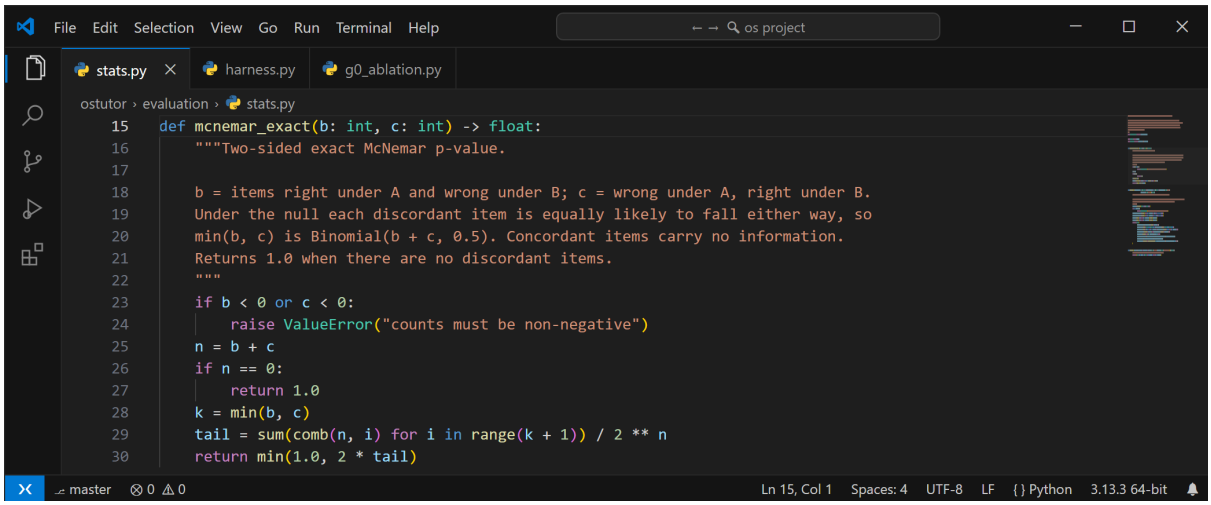


Figure 14: `ostutor/evaluation/stats.py`: the exact two-sided McNemar test used for every p-value in Section 4.

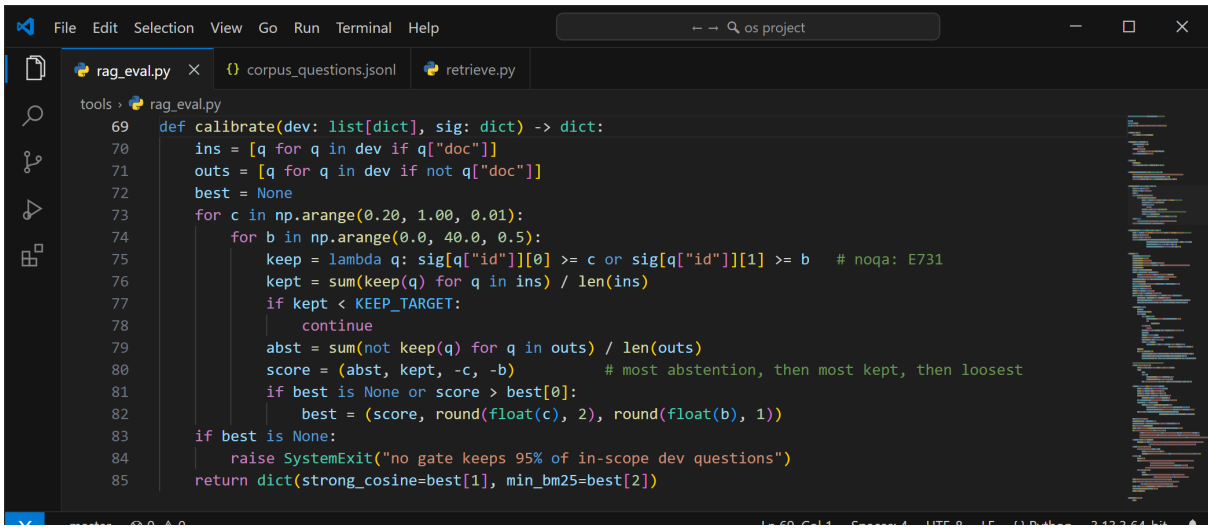


Figure 15: `tools/rag_eval.py`: the gate is fitted on the dev questions only. It must keep notes for at least 95% of in-scope questions, and among those settings it withholds notes from the most off-topic ones. It is then reported on the test half.

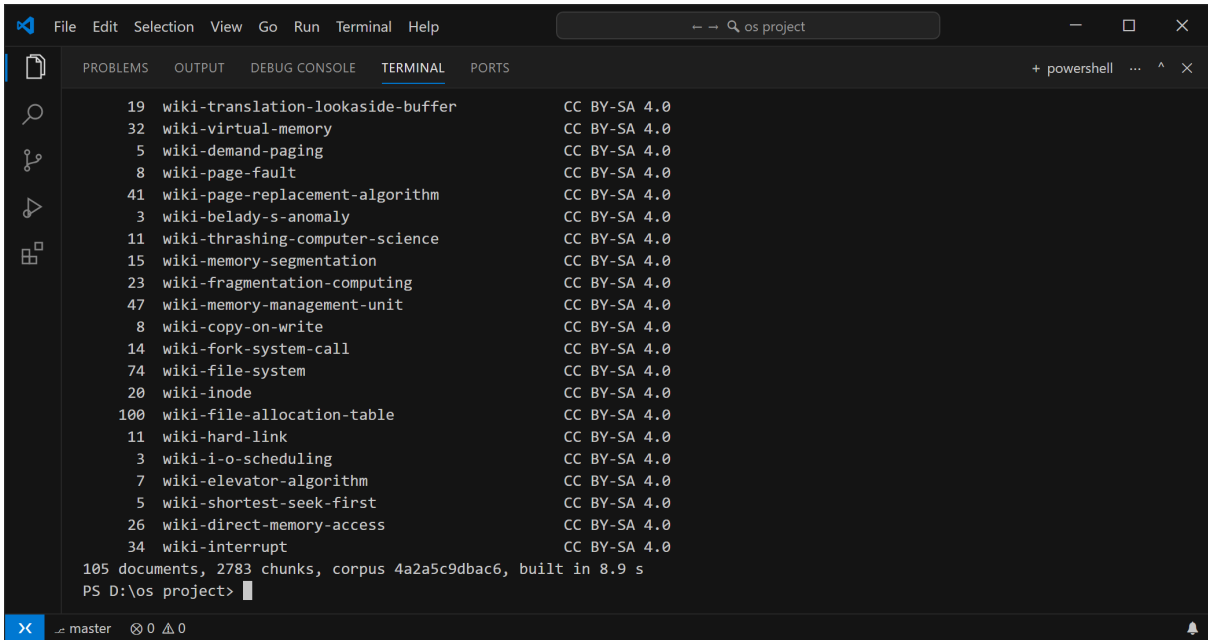


Figure 16: The end of `python -m ostutor.rag.ingest data\rag\corpus`: chunks per document, then the corpus total.

Four test suites need no GPU: `tools/test_pipeline.py` (data pipeline), `tools/test_router.py` (28 labelled routing cases, 0 misroutes allowed on shown attempts), `tools/test_g0.py` (55 checks on statistics, extraction scoring, grader regressions and the one-

adapter guard), and `tools/test_rag.py` (65 checks). The gateway suite grew from 28 to 37 checks.


File Edit Selection View Go Run Terminal Help
os project
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
+ powershell
PS D:\os project> python tools\test_rag.py
corpus: manifest rules
PASS the fixture loads
PASS every chunk carries its note and section
PASS every chunk carries a licence
PASS no chunk exceeds the size limit
PASS chunk ids are unique
PASS the corpus hash is stable
PASS a document with no source is refused
PASS a document with no owner is refused
PASS a document with no added is refused
PASS a document with no sha256 is refused
PASS a document with neither licence nor permission is refused
PASS a permission alone is enough
PASS a file changed after it was recorded is refused (checksum)
PASS a file with no manifest entry is refused
PASS derived files under extracted/ are not documents
PASS headings nest into a section path
PASS a long section is split with overlap
ingest
PASS add records owner, source, date and checksum
PASS an added file then ingests
PASS add refuses a file with neither licence nor permission
extraction
PASS a slide's title becomes its section
PASS slide body text is kept
PASS a Word heading becomes a section
PASS Word tables are kept as rows
PASS each PDF page is a section
PASS large-font lines become headings
PASS running headers repeated on every page are dropped
PASS ligatures are folded so 'first' matches
bm25
PASS the matching document scores highest
PASS an unrelated query scores zero
PASS stop words are ignored
retrieval (BM25 only)
PASS finds the Belady section
PASS returns at most k chunks
PASS a query sharing no word with the notes returns nothing
retrieval (hybrid, local embedding model)
PASS a paraphrase finds thrashing
PASS an off-topic question abstains
PASS gate='none' never abstains
PASS the course corpus loads with its calibrated gate
PASS every course-corpus document says whether it is course or online material
prompt and citations
PASS no hits leaves the trained prompt untouched
PASS the trained prompt is kept word for word at the start
PASS each passage is labelled [S1], [S2]
PASS the model is told never to invent a citation
PASS the footer names each source
PASS no hits, no footer
PASS the course-notes wording is unchanged by default
PASS another corpus is not called the student's course notes
PASS the footer heading follows the corpus
PASS valid and invented citations are told apart
open corpus tools
PASS rag_compare: an alias counts only as a whole word
PASS rag_compare: aliases under 3 characters and bare numbers are ignored
PASS rag_e2e: an alias counts only as a whole word
PASS rag_e2e: aliases under 3 characters and bare numbers are ignored
PASS analyze_rag_pool: an alias counts only as a whole word
PASS analyze_rag_pool: aliases under 3 characters and bare numbers are ignored
PASS wikitext: templates and references removed, link text kept
PASS wikitext: content sections and subsections are kept
PASS wikitext: See also and External links sections are dropped
PASS amended grading: hyphen equals space, a bare number reply counts
PASS amended grading: still strict on wrong answers
PASS letter share counts letters only
PASS C code lines are counted as code
PASS short-answer set: 144 questions, half dev and half test
PASS short-answer set: no question contains its own answer
PASS a subset keeps the matching chunks with their own embeddings
ALL RAG TESTS PASSED
PS D:\os project>
master 0 0

Figure 17: `tools/test_rag.py`: 65/65 checks, covering the manifest rules, checksum and unlisted-file refusal, extraction of slides, Word and PDF, retrieval, the gate, the prompt and citations, and the open-corpus tools (answer grading, wikitext conversion, how passages are labelled).

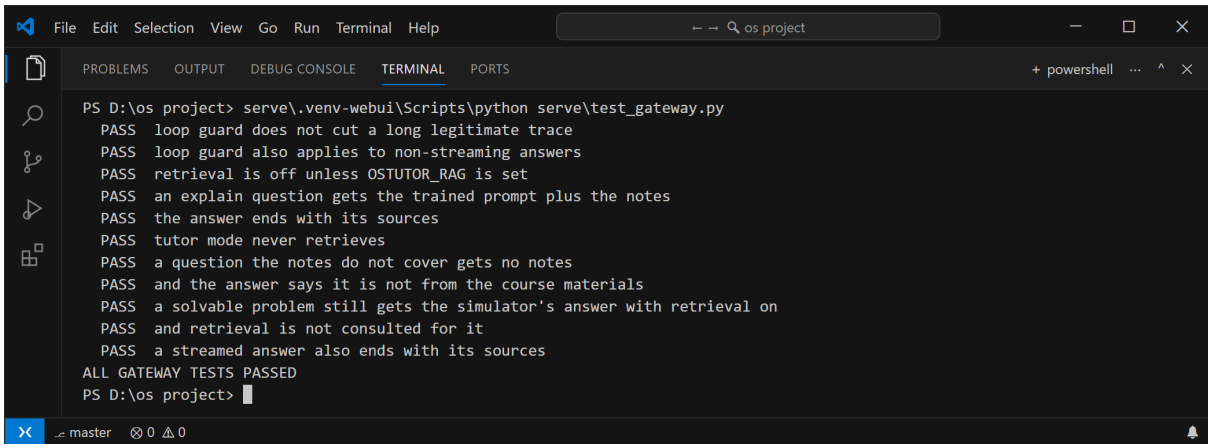


Figure 18: `serve/test_gateway.py` (last lines): the retrieval checks. Every earlier check is unchanged with retrieval off.

4. Metrics (Assessment): Results and Discussion

4.1 Metrics and why each was chosen

Metric	What it answers	Why this one
Simulator-exact accuracy	Is the final number right?	The simulator is ground truth; no judge model, no partial credit
Exact McNemar p	Is a before/after difference real?	Paired items; small discordant counts
Right→wrong / wrong→right	Did an unchanged mean hide cancelling losses?	Forgetting can hide inside a flat average
Extraction accuracy, delegation rate	Did the model read the problem correctly, and how often did the simulator answer?	Only answer-changing fields are scored (e.g. the quantum only for RR)
Validation / OOD loss	Did training fit without diverging?	A training-health signal, *not* a correctness measure (Section 4.2)
Precision / recall (fact checker)	Does a correction fire when, and only when, it should?	False alarms damage trust; misses ship misconceptions
Section recall@k, MRR@10	Did retrieval find the passage that answers the question?	Section-level is stricter than document-level, and a citation needs the right section
Abstention rate	Does retrieval return nothing when the notes do not cover the question?	A wrong passage is cited as support
Citation rate, invented citations	Does the answer use and honestly cite the notes?	A citation to a note that does not exist is the RAG equivalent of a hallucination
Latency p95, tokens/s	Does it stay fast?	Brief: retrieval ≤ 500 ms p95; generation ≥ 20 tok/s

Why no AUC-ROC or F1 over the whole system. The tutor is not a scored binary classifier, so there is no threshold to sweep. Precision and recall are used where a binary decision exists: the fact checker and the abstain gate.

4.2 Training loss

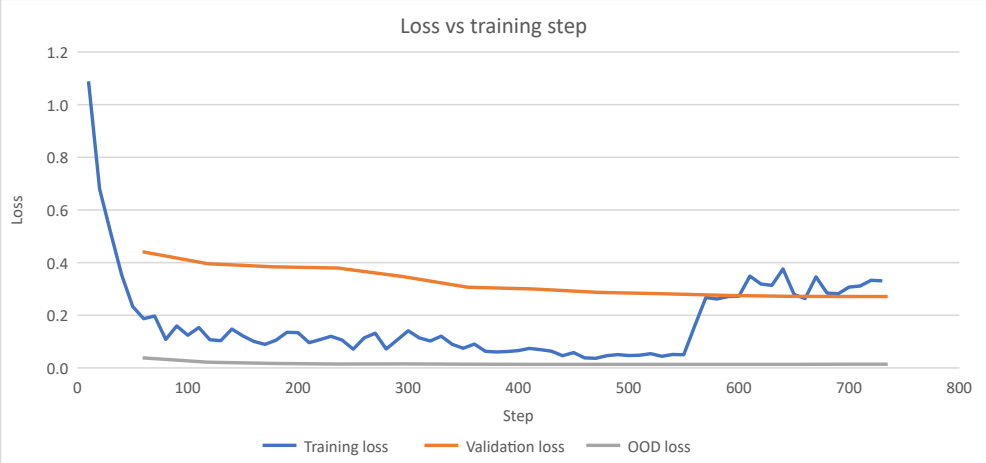


Figure 19: v3 training run, from the trainer's own log (`outputs/ostutor-lora-v3/ostutor-lora/train_metrics.json`).

Quantity	Start (step 59)	End (step 735)
Validation loss (240 rows)	0.4413	**0.2713**
OOD loss (logged, never used for selection)	0.0385	0.0139
Training loss, epoch average	0.1773 (last logged batches 0.31–0.33)	

The validation loss fell at every one of 12 evaluations, so training was healthy. The training loss climbs from near zero to 0.31–0.33 after step ~550. The training order is engineered (rectifier interleaving, spaced replay), so the late batches have a different mix from the early ones; this was not investigated further, and validation kept falling through it. It does **not** show the model can solve problems. The OOD loss is low because that split is fully templated procedural text, yet the same model solves only 34 of 57 suite problems on its own. Teacher-forced loss measures how well the model predicts a correct answer it is shown, not whether it can produce one.

4.3 Procedural accuracy and ablation

All six configurations were measured on the served path: llama.cpp with one adapter per server, greedy decoding, 57 items. They were graded with the corrected grader (D-003 and D-004, Section 5).

Configuration	Correct	Accuracy	Delegated to simulator
Base (untuned)	30/57	52.6%	0
Base + solver	49/57	86.0%	38
v2.2 adapter	19/57	33.3%	0
v2.2 + solver	43/57	75.4%	38
v3 adapter	34/57	59.6%	0
v3 + solver (deployed)	**57/57**	**100.0%**	**38**

Source: `reports/g0/20261012-1429/regraded.md`. As originally graded, base was 25/57; the 5 changed items are listed there.

What is being tested	A → B	Right→wrong	Wrong→right	p	Verdict
Solver, on v3	34 → 57	0	23	$< 10^{-6}$	significant
Solver, on the base	30 → 49	0	19	4×10^{-6}	significant
Adapter, with the solver	49 → 57	0	8	0.0078	significant
Adapter, without the solver	30 → 34	11	15	0.557	not significant
v2.2 vs base, no solver	30 → 19	17	6	0.035	v2.2 significantly worse
v2.2 → v3, no solver	19 → 34	2	17	0.0007	significant

Reading the ablation. The solver contributes most of the accuracy: +19 to +23 items, with zero losses. The adapter's measurable value is the 8 items the solver cannot take on its own (49 → 57, no losses). Every one of those 8 comes from the adapter answering the problems the solver declines. Without the solver, the adapter's +4 is indistinguishable from noise. v2.2 actively hurt the model, which is why v3 was rebuilt.

4.4 Out-of-distribution holdout

Kind	n	Extraction	Delegated	Correct	Delegated but misread
Banker's request	91	98.9%	98.9%	100.0%	0
Banker's safety	96	96.9%	96.9%	100.0%	0
Disk scheduling	80	88.8%	100.0%	92.5%	9
Free-space allocation	74	100.0%	100.0%	100.0%	0
Page replacement	86	98.8%	100.0%	100.0%	1
CPU scheduling	91	98.9%	98.9%	98.9%	0
Address translation	82	100.0%	100.0%	100.0%	0
All	**600**	**97.5%**	**99.2%**	**98.8%**	**10**

Source: `reports/g0/20261012-1429/ood.json`, `summary.md`. v3 + solver, end to end.

7 of the 600 answers were wrong. Six were disk problems where the stated range "0–299" was read as size 299 instead of 300 cylinders. The re-extraction diagnostic reproduced this off-by-one in **9 of 9** misread items. Grounding cannot catch it, because 299 *does* appear in the question. The seventh wrong answer was a scheduling problem that was not delegated.

4.5 Catastrophic forgetting

Benchmark (paired)	Base	v3	Δ	R→W	W→R	p
General MMLU (400)	80.25%	79.50%	−0.75 pp	13	10	0.678
Adjacent CS (300)	64.67%	67.33%	+2.66 pp	11	19	0.201
OS theory MCQ (70)	90.00%	85.71%	−4.29 pp	4	1	0.375

Source: `reports/forgetting_v3_paired.json`; p-values from `reports/g0/historical_significance.md`.

No effect is statistically detectable in either direction. That covers both the OS-theory dip and the adjacent-CS gain. The 70-item theory set is too small to decide a 4-point change, because the result hinges on 5 discordant questions; a 200-item set is planned. The base model scores identically, item for item, in two separate runs, so this evaluation is deterministic.

4.6 Fact checker

Ten probes were run, one per correction rule, and every v3 answer was read by hand. The checker produced **1 true positive, 2 false positives, 1 false negative and 6 true negatives**, so precision is 1/3 and recall is 1/2. Both false alarms were *correct* negated sentences: the rules match "suffers from Belady's anomaly" without seeing the "does not" before it. The miss was `seg_internal_frag`. On a broader set of 70 theory answers it flagged 0. Source: `reports/g0/20261012-1429/factcheck_v3.json`.

4.7 RAG: retrieval quality and latency

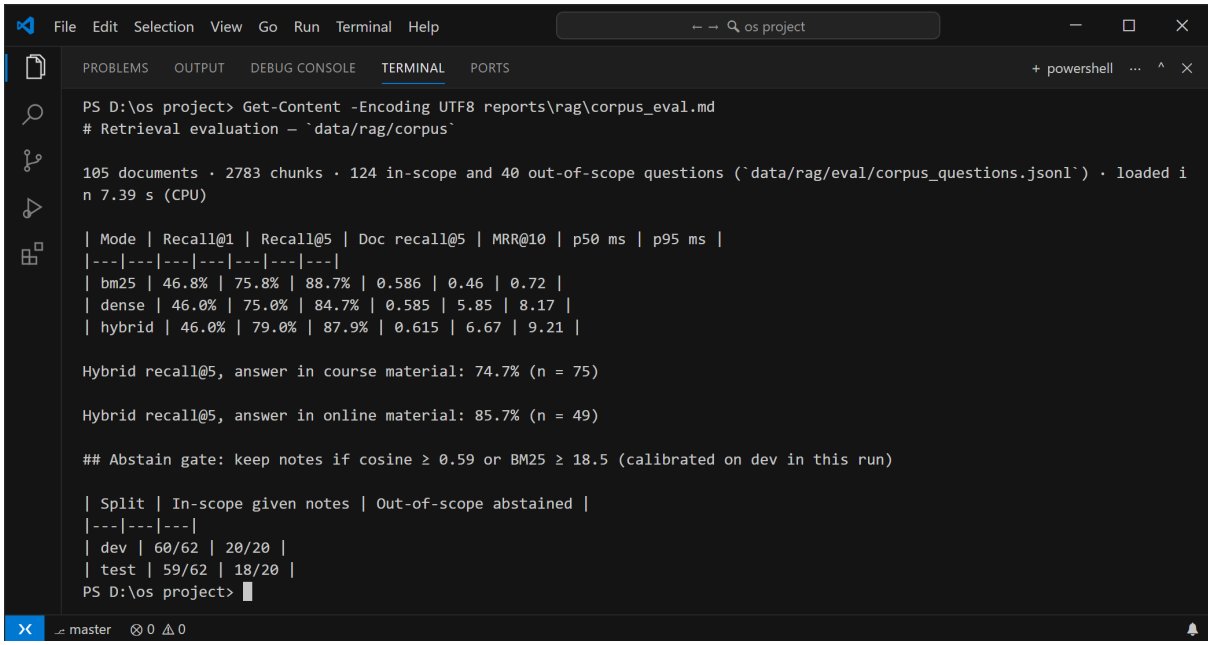


Figure 20: `tools/rag_eval.py` over the course corpus: 124 in-scope and 40 out-of-scope questions, 5 timed passes each (the first discarded).

Retrieval	Recall@1	Recall@5	Document recall@5	MRR@10	p95 latency
BM25 only	46.8%	75.8%	88.7%	0.586	0.7 ms
Dense only (MiniLM)	46.0%	75.0%	84.7%	0.585	7.8 ms
Hybrid (served)	**46.0%**	**79.0%**	**87.9%**	**0.615**	**8.3 ms**

Source: `reports/rag/corpus_eval.md`. A hit is a chunk from the labelled document that contains the labelled evidence phrase. Latency is CPU-only, per query.

"Not covered" gate (cosine ≥ 0.59 or BM25 ≥ 18.5)	In-scope questions given notes	Out-of-scope questions given none
Dev half (used to fit the gate)	60/62	20/20
Test half (not used for fitting)	59/62	18/20

- **Recall@5 is 79.0%, and 74.7% when the answer is in the course material** (85.7% for the online texts). The measure counts only the one labelled passage. Many misses retrieved a different document that also answers the question, for example the Wikipedia article on process states instead of the lecture slide on them. Document-level recall is 87.9%.
- **A larger embedding model did not help significantly.** multilingual-e5-base raised hybrid recall@5 to 81.5%. Paired, MiniLM missed 11 questions that e5 found and e5 missed 8 that MiniLM found (exact McNemar $p = 0.65$). e5 also costs three times the latency (25 ms p95), a 5.5-minute index build and about 1 GB more memory, so MiniLM was kept (`reports/rag/corpus_multilingual-e5-base_eval.md`).
- **The gate generalises to the held-out half.** It withheld notes from 18 of 20 off-topic questions and gave them to 59 of 62 in-scope ones. The thresholds had to be refitted for this corpus: the value fitted earlier on our 12 test notes (BM25 ≥ 6.0) would have given notes to 37 of the 40 off-topic questions, because BM25 scores are larger on a larger corpus.
- **Latency is far inside the 500 ms budget** at 8.3 ms.

Open corpus against the course corpus. The two corpora have no labelled passages in common, so they were compared by *answer recall*: a top-k chunk contains an accepted answer to one of the 144 short-answer questions. The selection rule was fixed in advance: the compiled corpus is changed only if some variant beats it on the dev half with $p < 0.05$. No variant did, so the test half was used once, for reporting.

Corpus	Chunks	Recall@5, dev	Recall@5, test	Recall@10, test	Recall@5, all 144
Course corpus	2,783	75.0%	73.6%	76.4%	74.3%
Open pool, unfiltered (control)	21,715	83.3%	81.9%	94.4%	82.6%
Open corpus, compiled (served)	**18,515**	**83.3%**	**83.3%**	**94.4%**	**83.3%**
without Wikipedia	8,628	65.3%	65.3%	79.2%	65.3%
without Stack Exchange	11,346	79.2%	84.7%	93.1%	81.9%
without Hailperin	17,688	87.5%	81.9%	93.1%	84.7%

Source: `reports/rag/compare_dev.md`, `compare_test.md`, `compare_all.md` (`tools/rag_compare.py`). Hybrid retrieval, no gate.

- **The open corpus retrieves the answer more often.** On the test half it gained 12 questions and lost 5 against the course corpus (83.3% against 73.6%, $p = 0.143$, not significant at $n = 72$). Over all 144 questions it gained 21 and lost 8 ($p = 0.024$). At recall@10 the gap is wider: 94.4% against 76.4% on the test half.
- **Wikipedia does most of the work.** Without it recall@5 falls to 65.3% ($p = 0.002$ on the test half). No other source's removal makes a significant difference.
- **The filters cost nothing.** The unfiltered pool scores the same within one question while being 17% larger. The filters remove text, not answers.
- **A tempting change that did not hold.** On the dev half, removing Hailperin's book scored best (87.5%). It was not significant ($p = 0.25$), so the rule kept the book, and on the test half the variant scored 81.9%, below the compiled corpus.

- **The gate abstains less.** Refitted on the dev half (cosine ≥ 0.52 or BM25 ≥ 24.0), it gave notes for 72 of 72 in-scope test questions but withheld them for only 14 of 20 off-topic ones (course corpus: 18 of 20). The six that leaked are on neighbouring subjects the wider corpus touches: B-trees, the CAP theorem, database normalisation, HTTP/2, monads and polymorphism (`reports/rag/open_gate.json`).
- **Latency** is 26 ms at the 95th percentile on the CPU, against 8 ms for the course corpus: still far inside the budget.

4.8 RAG: end to end on the served model

All of these were measured on the served model: one adapter per server, greedy decoding, prompt cache off.

A. Theory MCQ, 200 items. The model is asked for the option letter, and retrieval uses only the question text, as a student would type it. This generated-letter protocol differs from the log-likelihood one behind the 90.00% and 85.71% in Section 4.5, so the base model was measured again here.

Configuration	Correct	Accuracy	On the 130 new items
Base model, no retrieval	185/200	92.50%	121/130
Base model, with retrieval	186/200	93.00%	121/130
Fine-tuned v3, no retrieval	186/200	93.00%	121/130
Fine-tuned v3, with retrieval (served)	**184/200**	**92.00%**	**120/130**

Comparison	Right→wrong	Wrong→right	exact McNemar p
Base → v3 with retrieval	5	4	1.000
v3 → v3 with retrieval	6	4	0.754
Base → base with retrieval	3	4	1.000
Base → v3 (no retrieval)	1	2	1.000

Source: `reports/rag/e2e-20261012-2239/summary.md`. The gate gave notes for 152 of the 200 questions.

B. Explain answers. The 62 test-half questions were answered by v3 with retrieval on.

Measure	Result
Questions given notes by the gate	59/62
Notes included the labelled document	46/59
Answers citing at least one note	50/59
Invented citations (an [S*n*] with no such note)	0
Answers flagged by the fact checker	0
Median prompt / answer length	816 / 262 tokens
Median time to the full answer	6.5 s
Decode speed, median / slowest (brief: ≥ 20 tok/s)	42.9 / 41.6 tok/s

C. Faithfulness. Every sentence carrying a citation was checked against the passage it cites: 143 sentence–passage pairs.

Check	Supported
Base-model judge, all 143 pairs	117/143 (81.8%)
Our own check of a random 40 of them, strict rule	24/40 (60%)
Base-model judge on the same 40	29/40 (72.5%)

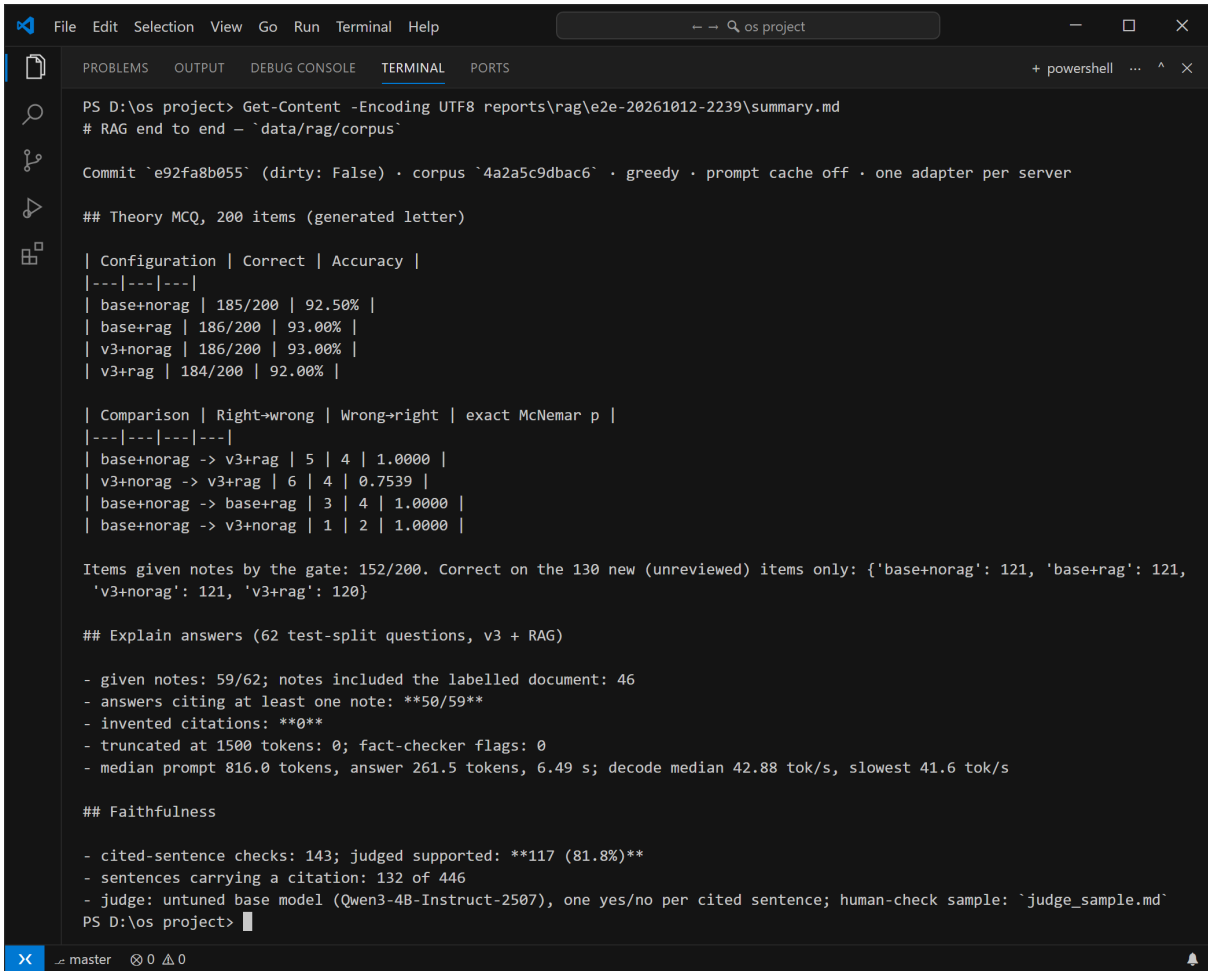


Figure 21: The run's own summary file, `reports/rag/e2e-20261012-2239/summary.md`.

Reading the results.

- **The MCQ target was not met, and retrieval made no measurable difference.** The target was "v3 with retrieval ≥ base"; it scored 184 against 185, and the difference is one question ($p = 1.0$). All four configurations are within two questions of each other. The base model already answers 92.5% of these questions correctly without notes, so there is little left for notes to fix. On this larger set the fine-tuned model is also no worse than the base, unlike the 70-item log-likelihood result.
- **Citations are honest but sparse.** No answer cited a note that does not exist, and 50 of 59 answers cited at least once. But only 132 of the 446 answer sentences carry a citation. The rest are the model's own explanation, which the student cannot check against the notes.

- **About 60% of cited sentences are fully supported** on our strict reading: everything the sentence says must be in the cited passage. The usual failure is a correct quotation followed by the model's own elaboration in the same sentence, or an example the passage does not contain. One answer quoted a passage accurately and then added a worked address example with wrong arithmetic. The base-model judge is too lenient: where we disagreed with it, it had said "supported" 8 times and we had said so 3 times. So its 81.8% overstates faithfulness.
- **Speed is unaffected.** A median prompt of 816 tokens, most of it retrieved notes, did not slow decoding (42.9 tok/s against the 20 tok/s floor). These are server-reported decode rates and exclude prompt processing, which is why they differ from the 26.9 tok/s wall-clock figure measured earlier through the full stack.

D. Short answers, with each corpus. The 144 short-answer questions were put to the base model and to v3, with retrieval off and on, under the explain-mode prompt and the instruction to answer in a few words. An answer is correct when it contains an accepted answer as a whole word. The course-corpus and open-corpus runs gave identical replies on all 288 retrieval-off items, so the two runs compare like for like.

Configuration	Test half (72)	All 144
Base model, no retrieval	48 (66.7%)	100 (69.4%)
Base model + course corpus	59 (81.9%)	122 (84.7%)
Base model + open corpus	62 (86.1%)	124 (86.1%)
Fine-tuned v3, no retrieval	49 (68.1%)	102 (70.8%)
Fine-tuned v3 + course corpus	59 (81.9%)	124 (86.1%)
Fine-tuned v3 + open corpus	**62 (86.1%)**	**123 (85.4%)**

Comparison (v3)	Right→wrong	Wrong→right	exact McNemar p
No retrieval → open corpus, test half	4	17	0.0072
No retrieval → open corpus, all 144	9	30	0.0011
No retrieval → course corpus, test half	4	14	0.0309
Course corpus → open corpus, test half	4	7	0.549
Course corpus → open corpus, all 144	11	10	1.000

Source: `reports/rag/e2e-20261013-1339/summary.md`, `reports/rag/e2e-short-course/summary.md`, `reports/rag/short_e2e-short-course_vs_e2e-20261013-1339.md`.

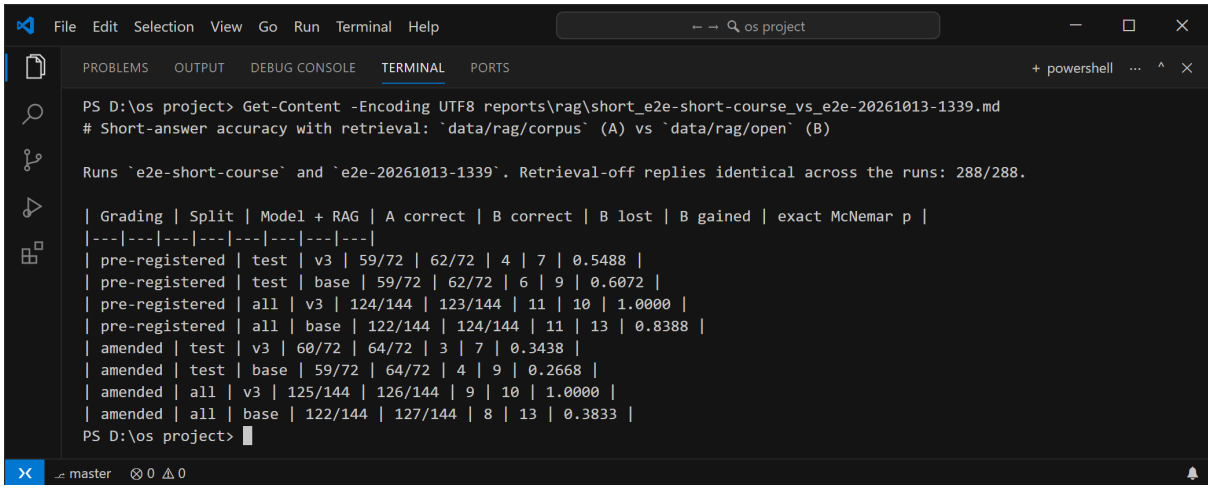


Figure 22: `tools/rag_e2e_compare.py`: the course corpus (A) against the open corpus (B), item by item, under both gradings.

E. The open corpus on the other measures. On the 200 MCQ items v3 scored 186 with and without the open corpus, and the base model 185 without and 184 with it (all p = 1.0). For explain answers, the 72 test-half short-answer questions were asked again for a full explanation. **69 of 72 answers cited at least one passage**, none cited a passage that does not exist, and the decode speed was 42 tok/s (slowest 39). The judge rated 110 of 138 cited sentences supported (79.7%). Our strict check of a random 40 found 25 of 38 claims fully supported (two lines were bare "Reference: [S2]" lists and were not counted), close to the 24 of 40 for the course corpus. The judge was again lenient: 8 disagreements in its favour and 1 against.

Reading these results.

- **Retrieval clearly raises short-answer accuracy**: by 13 to 14 questions out of 72 on the test half, significant for both corpora. This is the measurable accuracy gain the MCQ could not show. On short answers the model must produce the term itself and cannot recognise it among four options.
- **The better corpus did not give better answers.** v3 answered 62 and 59 test questions with the open and course corpora, and 123 and 124 of all 144. The difference is nowhere near significant. The reason shows when each answer is split by whether its passages contained the answer. Retrieval rescued a wrong answer 26 times with the open corpus and 25 times with the course corpus. Most of the open corpus's extra hits were for questions the model already answered correctly without help (85 against 66). And off-target passages turned a right answer wrong 7 times with the open corpus, against 3 with the course corpus.
- **The open corpus meets the citation target** (69/72, against a target of 90%). The course-corpus figure of 50/59 was measured on a different question set, so the two rates are not directly comparable.

4.9 Discussion

- **Each layer guards a different failure.** The simulator guards procedures: a 4B model cannot execute a long schedule reliably, but it can read one. Retrieval ties declarative claims to passages the student can check. The fact checker guards a known list of misconceptions. Tutor mode is shielded from all three, because its job is to *withhold* the answer.
- **Fine-tuning's measured value is narrower than first claimed.** With the solver, the adapter adds 8 significant items. Without it, there is no significant gain over the untuned base on the problem suite, and none on the 200-item theory set. Its other contributions are the trained output formats, the Socratic behaviour and the shell mode, and none of these is yet measured by a number (G3).
- **Retrieval's value is accuracy on recall questions, plus traceability.** It did not raise MCQ accuracy, which was already near its ceiling. It did raise short-answer accuracy by about 15 points, with either knowledge base ($p \leq 0.001$ over all 144 questions). It also ties each answer to a named source and warns when nothing relevant was found. Whether that helps students learn is not measured yet.
- **More and better data stopped paying off at the model.** The open corpus was built with care and retrieves the answer more often (83% against 74%). But the model already knows most of what the extra passages contain, and the wider corpus brings more distracting passages. So the choice between the two corpora is about what the student should see, not about accuracy. The course corpus cites the course's own slides and says "not covered" more often. The open corpus covers more ground and can be shared.
- **Measurement found more errors than the model did.** Two grader blind spots, a non-deterministic cache, a void benchmark, two unsupported claims and a lenient faithfulness judge all biased the numbers in the project's favour. Each was found by auditing the evaluation rather than the model (Section 5).

4.10 Targets and results

These targets were set before the course-corpus run, partly by the brief and partly by us.

Measure	Target	Measured	Met?
Retrieval p95 latency	≤ 500 ms	8.3 ms	Yes
Labelled question set	≥ 100 items	124 + 40	Yes
Recall@5 (passage level)	≥ 85%	79.0%	No
Off-topic questions given no notes (test half)	≥ 90%	18/20 = 90%	Yes, at the limit
Answers citing a note, when given notes	≥ 90%	50/59 = 85%	No
Invented citations	0	0	Yes
Theory MCQ (200 items), v3 with retrieval	≥ base	92.0% vs 92.5%	No (one question; p = 1.0)
Faithfulness: human check of ≥ 30 claims	done	sample prepared; our check 24/40	Pending the instructor's check
Generation speed with notes in the prompt	≥ 20 tok/s	41.6 tok/s slowest	Yes
Procedural suite, full system	57/57	unchanged: retrieval never runs for solvable problems	Yes (by design; tested)

The same targets for the open corpus (the recall measure here is answer recall on the short-answer test half, not passage recall):

Measure	Target	Measured	Met?
Retrieval p95 latency	≤ 500 ms	26 ms	Yes
Recall@5 (answer recall, test half)	≥ 85%	83.3%	No
Off-topic questions given no notes (test half)	≥ 90%	14/20 = 70%	No
Answers citing a passage, when given passages	≥ 90%	69/72 = 95.8%	Yes
Invented citations	0	0	Yes
Theory MCQ (200 items), v3 with retrieval	≥ base	93.0% vs 92.5%	Yes, numerically (p = 1.0)
Short answers, v3 with retrieval vs without (test half)	better	86.1% vs 68.1%	Yes (p = 0.007)
Generation speed with passages in the prompt	≥ 20 tok/s	39.2 tok/s slowest	Yes

5. What Went Wrong

These are listed by impact. Each entry gives what happened, how it was found, and what was done about it. The first group distorted *reported numbers*, which is the most serious kind of error for a project whose claims rest on measurement.

5.1 Errors in measurement and reporting

What went wrong	Effect	Fix / status
Void benchmark: v2.2 and v3 loaded into one llama-server and switched by per-request LoRA scale	Did not isolate the adapters: v3 scored 17/57 there and 36/57 alone, so every column of that benchmark was wrong	Declared void. One adapter per server, enforced by `check_adapters`, which refuses to measure otherwise
Unsupported claims in submitted PDFs: "final training loss 0.2710", and "the flips confirm genuine transfer"	0.2710 appears in no training artifact. The transfer has p = 0.20	Withdrawn in `PROJECT_CONTEXT.md` and in this report. The two earlier PDFs still need correcting (owner decision)
Grader blind spots (D-003, D-004): LaTeX `\quad` separators, and a "Final Answer" line with a bare number	5 correct base-model answers were graded wrong. That inflated the adapter's apparent no-solver gain from p = 0.557 to p = 0.136	Fixed with regression tests. Re-grading 799 historical outputs changed 0 verdicts. Base-model failures are now hand-audited
Prompt cache made greedy decoding non-deterministic (D-005)	Same weights scored 36/57 and 34/57; only 23/57 outputs were identical	Evaluation sends `cache_prompt: false` (6/6 byte-identical repeats); serving keeps the cache. The deterministic rerun of G0 was interrupted and is still pending
Teacher-forced loss read as correctness	A low OOD loss (0.0139) was taken as evidence of generalisation	Loss is reported only as a training-health signal (Section 4.2)
Short-answer grading too literal	"execve" did not match "exec", "Context switching" did not match "context switch", and the answer "0" to "what does fork() return in the child" could never be graded correct, because bare digits are ignored	The pre-registered grading is reported as the result. A second grading is reported next to it: a hyphen counts as a space, an exact numeric reply counts, and four aliases are added. It was fixed from the base model's misses before any v3 answer was read, and applied to every configuration. It adds 2–4 answers to each and changes no conclusion

5.2 Errors in the system

What went wrong	Effect	Fix / status
Disk-size off-by-one in extraction ("0–299" read as 299)	6 confidently wrong OOD answers. Grounding passes it because 299 is in the text	Not fixed: a student-facing change awaiting owner approval. Proposed: read an "a–b" range as b − a + 1 cylinders
Fact checker ignores negation and misses one of its own rules	Precision 1/3, recall 1/2 on the probes	Not fixed; awaiting approval. Proposed: a negation window before each pattern, and probes for every rule in the test suite
Fact checker silently returned the misconception when its redraft call failed (D-002)	A student would see "LRU suffers from Belady's anomaly" with no warning	Fixed: the correction is attached. The test was also non-hermetic (it passed only with the server up)
Fact checker runs only on non-streaming requests	Open WebUI streams by default, so in the UI the checker may never run. Not confirmed: the gateway logs for this deployment are empty	To verify. Retrieval and its sources footer work on both paths
The first RAG abstain gate used dense similarity alone	On our test notes, 4 of 10 off-topic questions retrieved OS passages	Replaced by a two-signal gate; its thresholds are now fitted per corpus on held-out questions
Thresholds fitted on one corpus do not carry over	The test-note thresholds would have given notes to 37 of the 40 off-topic questions on the course corpus	Refitted on the dev half (18.5); stored with the corpus in `gate.json`
OSTEP running headers inside passages	"OPERATING SYSTEMS [VERSION 1.01] WWW.OSTEP.ORG" appeared mid-sentence in 11 of the 27 sampled OSTEP passages	Lines repeated on a third of the pages are dropped
Ligatures in typeset PDFs	"file" never matched a query for "file"	NFKC normalisation
Sparse citations and a lenient judge	Only 132 of 446 answer sentences cite a note; the base-model judge rates 81.8% supported where our strict check finds about 60%	Not fixed. Proposed in Section 6.4
Wikipedia reference sections kept in the first open-corpus build	1,137 of 19,674 chunks were "See also", link and reference lists, and they appeared in answers' sources	A heading-level off-by-one; fixed, tested and rebuilt (Section 2.5)
Open-corpus passages labelled "course notes"	A student would have been told that Wikipedia and Stack Exchange text came from their course notes	Each corpus now names its passages in its manifest. The open corpus shows "Sources (openly licensed references, not your course notes)"; the course corpus wording is unchanged and tested word for word
One legacy Word file could not be converted	Word automation waited on an invisible dialog; the lab handout is missing from the corpus	Recorded as excluded; it can be added by saving it as .docx

5.3 Errors in the data pipeline and training

What went wrong	Effect	Fix
More data was used to fix scheduling	Scheduling had the most rows (1,733) and the worst accuracy (42%). All 7 of 7 failures had perfect arithmetic and a wrong schedule	Diagnosed as architectural; fixed by delegation, not more data (64.9% → 100%)
`clean.py` required ≥ 4 messages for tutoring	Silently discarded all 1,508 OS tutoring rows	Threshold < 2, plus a release gate requiring ≥ 300 OS tutoring rows
Naive 13-gram decontamination	Removed 58% of procedural rows (shared template text)	Distinctive n-grams only, plus parameter fingerprints
Decontamination keyed on `category == "procedural"`	Removed 58% of OS tutoring rows	Keyed on whether a row has a fingerprint
`numbers_in()` missed a number followed by "."	Grounding wrongly rejected valid extractions	Fixed with tests
v2.2's second epoch	Memorised more; v2.2 is significantly *worse* than base ($p = 0.035$)	v3 trains one epoch
A `memory_probe()` before training	Bypassed Unsloth's checkpointing and reported a false OOM	Replaced by running the longest rows first through the real trainer

5.4 Errors in engineering process

- **A benchmark chain waited an hour polling itself.** `busy("benchmark.py")` matched `run_full_benchmark.py`. Polling for a free GPU between jobs also raced and crashed the next job. Fixed: match the full path, and require several consecutive clear polls.
- **Git normalised 57 CRLF files to LF.** The repository was staged before line-ending conversion was disabled, which would have silently changed bytes the dataset manifest hashes. Fixed and verified file by file (D-001).
- **The evaluation questions are ours.** The retrieval questions and the new MCQ items were written by the team, not by the instructor, and were written while looking at the passages. We tried to phrase them as students would, but they are likely easier than real student questions.
- **The GPU is shared.** Another training job on the same machine started four minutes after our end-to-end run and took most of the card's memory, and our model server crashed out of memory halfway through. The run resumed from its saved rows once the card was free. We now wait for ten idle minutes before starting, and stop if the other job reappears.
- **Wikipedia article identifiers lost characters.** "Bélády's anomaly" became `wiki-b-l-dy-s-anomaly` and "Producer–consumer" lost its dash, which broke two question labels. Titles are now transliterated before the identifier is made.

6. Alternative Flow (Improvements)

For each stage: what we would do differently with hindsight, or next, and the evidence that motivates it.

6.1 Evaluate before training, not after

The biggest lesson is ordering. The project trained three adapters before it measured the untuned base on the same suite with the same grader. Done the other way round, the ablation would have shown on day one that the solver, not more fine-tuning, was the lever. A better flow:

1. Build the evaluation sets and the grader, then audit the grader on the *base* model's outputs.
2. Measure the base model, deterministically, with a significance test for every comparison.
3. Add tools (solver, retrieval) and measure again.
4. Fine-tune only for the gaps tools cannot close, and state the expected effect size first.

6.2 Data

- **Targeted declarative replay** for the OS-theory topics that regressed, or a LoRA-scale sweep, so theory accuracy can recover to ≥ base without retraining everything.
- **Grow tutoring supply with genuine material.** Tutoring is the scarcest category and bounds the whole dataset. Rewordings would add memorisation pressure, not supply.
- **Harder theory questions.** The 200-item set now exists, but every configuration scores 92–93% on it, so it can no longer separate them. Questions that need reasoning across concepts, written or reviewed by the instructor, would show differences that this set cannot.

6.3 Model

- **Zero-shot probe newer bases** (Qwen3.5-4B, Gemma 4 12B, non-thinking) on the theory MCQ before any retraining, with Qwen3-4B-2507 as control.
- **Merge the LoRA before quantising.** Runtime LoRA is the likely reason decode speed is below the card's bandwidth bound. Speculative decoding with a 0.6B draft model is a second option.

6.4 Retrieval

- **Make the model cite every sentence it takes from the notes.** The prompt currently asks for citations "where the answer relies on the notes"; a stricter instruction, or a short worked example of a fully cited answer, should raise the 132-of-446 coverage. The change should be compared on the dev questions before being reported on the test half.
- **Check answers against their sources before showing them.** Split the answer into sentences and drop or flag any cited sentence its passage does not support. A stronger judge than the 4B base model is needed for that: our check found the current one too lenient.
- **Questions from the instructor and from students.** Real questions, labelled by the instructor with every passage that answers them (not just one), would give a fairer recall figure than our single-passage labels.

- **A re-ranker rather than a bigger embedder.** A larger embedder bought 2.5 points of recall that were not significant. A small cross-encoder that re-scores the top 20 attacks the actual weakness, which is ranking the right passage first (recall@1 is 46%).
- **Learned or calibrated abstention.** Use a logistic model over (max cosine, max BM25, margin between the top two) instead of hand-set thresholds, with precision and recall reported on held-out data.
- **Grounding checks on the answer, not just the prompt.** Verify that each cited sentence is supported by the cited passage (entailment, or overlap as a first step). The fact checker becomes a check against the notes instead of a fixed rule list.
- **Spend the effort where retrieval fails, not on more text.** A larger, cleaner corpus raised answer recall but not answer accuracy (Section 4.8). Of v3's 21 wrong answers with the open corpus, 16 came with four passages that did not contain the answer, although the corpus holds an answer to nearly every question. That is a ranking problem. In the other 5 the answer was in front of the model and it chose a near miss: "dirty bit" for "reference bit", "Access Control List" for "capability list". A re-ranker, and a check that the answer is actually stated in a cited passage, address both.
- **Treat Stack Exchange questions differently from answers.** A question body often states the asker's misconception. It helps matching, because students phrase questions the same way, but it should not be offered to the model as a source to cite. Indexing questions for search and citing only answers would keep the benefit without the risk.
- **A gate per source.** The open corpus lets more off-topic questions through (14/20 withheld against 18/20) because Stack Exchange covers neighbouring subjects such as networking and databases. Separate thresholds for the textbook, encyclopedia and forum parts would tighten it.
- **Retrieval for tutor hints, but behind a leak filter.** Passages could make Socratic hints more accurate, but only if a filter guarantees no passage states the final answer. That needs its own answer-leak measurement first (G3).

6.5 Verification and serving

- **Fix the disk off-by-one** and add a range-semantics check to grounding, not just a presence check.
- **Negation-aware fact checking**, with every rule exercised by a probe in the test suite.
- **Run the fact checker on streamed answers**: buffer prose answers, or check and append a correction after the stream ends.
- **Extend delegation to the 11 formula kinds**, which are still answered by the model unaided.

6.6 Process

- Every reported number names its file, and documents are generated from those files rather than typed by hand. The two incorrect PDF claims would not have been possible.
- Version control from the first day, with byte-exact storage and a commit hash in every report.
- Every evaluation run records its provenance (commit, model hashes, server arguments, decoding settings, cache setting), as the G0 harness now does.

Appendix A Code listings

This appendix gives the code shown in the figures of Section 3 as text, with the same line numbers.

A.1 build stages `ostutor/build.py : 145–171`

```
145
146     # 3 ------------------------------------------- clean / scrub / repair
147     step(3, "clean / scrub / repair")
148     rows = [r for v in buckets.values() for r in v]
149     rep = {}
150     rows = clean_records(rows, shell_blocklist=blocklist, report=rep)
151     M["stages"]["clean"] = rep
152     print(f"    kept {rep['kept']}/{rep['input']} | repaired "
153           f"{sum(rep['repaired'].values())} | PII redactions "
154           f"{sum(rep['pii_redactions'].values())}")
155
156     # 4 ---------------------------------------------- exact + lexical dedup
157     step(4, "deduplicate (exact + lexical)")
158     rep = {}
159     rows = dedup.exact(rows, rep)
160     rows = dedup.lexical(rows, rep)
161     M["stages"]["dedup"] = rep
162     print(f"    exact -{rep['exact_removed']} | lexical -{rep['lexical_removed']} "
163           f"| {len(rows)} remain")
164
165     # 5 ------------------------------------------------------- decontaminate
166     step(5, "decontaminate vs held-out eval + OOD")
167     rep = {}
168     rows = dedup.decontaminate(rows, eval_texts(), rep,
169                                eval_fingerprints=eval_fingerprints())
170     M["stages"]["decontaminate"] = rep
171     print(f"    removed {rep['removed_total']} {rep['removed']} | {len(rows)}
remain")
```

A.2 decontaminate `ostutor/dedup.py : 178–192`

```
178     distinctive = {g for g, c in df.items() if c <= boilerplate_df}
179     kept, hits = [], Counter()
180     fp_hits = 0
181     for r, s in zip(records, shared):
182         if s is None:
183             if r["lineage"]["fingerprint"] in eval_fingerprints:
184                 hits[r["category"]] += 1
185                 fp_hits += 1
186             else:
187                 kept.append(r)
188             continue
189         if s & distinctive:
190             hits[r["category"]] += 1
191         else:
192             kept.append(r)
```

A.3 lora `train/kaggle_train.py : 138–144`

```
138     model, tok = FastLanguageModel.from_pretrained(
139         BASE, max_seq_length=MAX_LEN, load_in_4bit=True, dtype=None)
140     model = FastLanguageModel.get_peft_model(
141         model, r=a.rank, lora_alpha=2 * a.rank, lora_dropout=0.05, bias="none",
142         target_modules=["q_proj", "k_proj", "v_proj", "o_proj",
```

```
                          "gate_proj", "up_proj", "down_proj"],
        use_gradient_checkpointing="unsloth", random_state=SEED)
```

A.4 sftconfig `train/kaggle_train.py : 172–199`

```
    cfg = SFTConfig(
        output_dir=a.out,
        dataset_text_field="text",
        max_length=MAX_LEN,
        packing=False,
        per_device_train_batch_size=per_device,
        gradient_accumulation_steps=accum,      # effective batch = 1 x 8 x world
        per_device_eval_batch_size=1,
        num_train_epochs=a.epochs,
        max_steps=a.max_steps,
        learning_rate=a.lr,
        lr_scheduler_type="cosine",
        warmup_ratio=0.03,
        weight_decay=0.01,
        max_grad_norm=1.0,
        optim="adamw_8bit",
        neftune_noise_alpha=5,
        fp16=True,
        bf16=False,
        eval_strategy="steps",
        eval_steps=every,
        save_strategy="steps",
        save_steps=every,
        save_total_limit=2,
        load_best_model_at_end=True,
        metric_for_best_model="eval_val_loss",
        greater_is_better=False,
        logging_steps=10,
```

A.5 grounding `ostutor/solver.py : 351–364`

```
            for y in x:
                out.extend(y if isinstance(y, list) else [y])
        elif isinstance(x, int):
            out.append(x)
    return [n for n in out if isinstance(n, int)]


# Values that must literally appear in the question. Derived quantities and defaults
# are excluded: a disk size of 200 or a priority of 1 is often implied, not written.
# `avail` is included deliberately: the one genuine misreading in testing was an
# invented Available vector ([10, 5, 7] for a problem stating [3, 3, 2]), which is
# precisely the error this check exists to catch. A question that gives TOTAL
instead
# of Available will fail grounding and fall back to the model answering for itself -
# declining is the safe direction.
```

A.6 corpus `ostutor/rag/corpus.py : 96–125`

```
 def validate(root: Path, manifest: dict) -> list[dict]:
     """Check every manifest entry and every file; raise CorpusError on the first
problem."""
     docs, seen, listed = manifest.get("documents", []), set(), set()
     for doc in docs:
         missing = [k for k in REQUIRED if not str(doc.get(k, "")).strip()]
         if not str(doc.get("licence", "")).strip() and not str(doc.get("permission",
"")).strip():
             missing.append("licence or permission")
         if missing:
```

```
104             raise CorpusError(f"document {doc.get('id', '?')}: no {',
'.join(missing)} "
105                               "recorded - refusing to index it")
106         if doc["id"] in seen:
107             raise CorpusError(f"duplicate document id {doc['id']}")
108         seen.add(doc["id"])
109         path = root / doc["path"]
110         if not path.is_file():
111             raise CorpusError(f"document {doc['id']}: {doc['path']} does not exist")
112         if path.suffix.lower() not in SUPPORTED:
113             raise CorpusError(f"document {doc['id']}: unsupported format
{path.suffix}")
114         if sha256(path) != doc["sha256"]:
115             raise CorpusError(f"document {doc['id']}: {doc['path']} does not match
its recorded "
116                               "checksum - refusing to index changed content")
117         listed.add(path.resolve())
118     unlisted = sorted(str(p.relative_to(root)) for p in root.rglob("*")
119                       if p.is_file() and p.suffix.lower() in SUPPORTED
120                       and not (set(p.relative_to(root).parts[:-1]) & NOT_DOCUMENTS)
121                       and p.resolve() not in listed)
122     if unlisted:
123         raise CorpusError(f"{len(unlisted)} file(s) have no manifest entry -
refusing to ingest: "
124                           + ", ".join(unlisted[:5]) + (" ..." if len(unlisted) > 5
else ""))
125     return docs
```

A.7 extract pdf `ostutor/rag/extract.py : 102–113`

```
102     # Running headers and footers ("OPERATING SYSTEMS [VERSION 1.01] WWW.OSTEP.ORG",
the
103     # chapter name in capitals, a copyright line) repeat on many pages and would be
glued
104     # into the middle of passages. A line on at least a third of the pages (min. 3)
is one.
105     seen = {}
106     for lines in pages:
107         for key in {_running(t) for _, t in lines}:
108             seen[key] = seen.get(key, 0) + 1
109     limit = max(3, len(pages) / 3)
110     running = {k for k, c in seen.items() if c >= limit and k}
111     parts = []
112     for n, lines in enumerate(pages, 1):
113         lines = [(s, t) for s, t in lines if _running(t) not in running]
```

A.8 retrieve `ostutor/rag/retrieve.py : 123–149`

```
123     def retrieve(self, query: str, k: int = 4, mode: str = "hybrid",
124                  gate: str = "combined") -> list[dict]:
125         """Top-k chunks as dicts {chunk, score, cosine}; [] when nothing is
relevant."""
126         if not self.chunks:
127             return []
128         rankings, cos, s = [], None, None
129         if mode in ("hybrid", "bm25"):
130             s = self.bm25.scores(tokens(query))
131             rankings.append([i for i in np.argsort(-s) if s[i] > 0])
132         if mode in ("hybrid", "dense") and self.model is not None:
133             q = self.model.encode([self.qp + query], normalize_embeddings=True)[0]
134             cos = self.emb @ q
135             rankings.append(list(np.argsort(-cos)))
136         if cos is not None and gate != "none":
137             if gate == "cosine" or s is None:
```

```
138                 if cos.max() < MIN_COSINE:
139                     return []
140             elif cos.max() < self.strong_cosine and s.max() < self.min_bm25:
141                 return []
142         fused: dict[int, float] = {}
143         for ranking in rankings:
144             for r, i in enumerate(ranking):
145                 fused[i] = fused.get(i, 0.0) + 1.0 / (RRF_K + r + 1)
146         best = sorted(fused, key=lambda i: -fused[i])[:k]
147         return [dict(chunk=self.chunks[i], score=fused[i],
148                      cosine=None if cos is None else float(cos[i])) for i in best]
149
```

A.9 augment `ostutor/rag/retrieve.py : 156–175`

```
156 NOUN = "course notes"
157 SOURCES_HEADING = "Sources from your course notes"
158
159
160 def augment(system: str, hits: list[dict], noun: str = NOUN) -> str:
161     """The system prompt with the retrieved notes appended and a citation rule.
162
163     The trained system prompt is kept word for word at the start, so the adapter
still
164     sees the prompt it learned the mode under; the notes follow it."""
165     if not hits:
166         return system
167     blocks = []
168     for n, h in enumerate(hits, 1):
169         c: Chunk = h["chunk"]
170         blocks.append(f"[S{n}] {c.citation()}\n{c.text}")
171     short = "notes" if noun == NOUN else noun
172     return (system + f"\n\n{noun[0].upper() + noun[1:]} that may be relevant:\n\n" +
"\n\n".join(blocks)
173             + f"\n\nWhere your answer relies on these {short}, cite them as [S1],
[S2] and so "
174               f"on. If the {short} do not cover the question, answer from your own
knowledge "
175               f"and say that the {short} do not cover it. Never invent a citation.")
```

A.10 calibrate `tools/rag_eval.py : 69–85`

```
69  def calibrate(dev: list[dict], sig: dict) -> dict:
70      ins = [q for q in dev if q["doc"]]
71      outs = [q for q in dev if not q["doc"]]
72      best = None
73      for c in np.arange(0.20, 1.00, 0.01):
74          for b in np.arange(0.0, 40.0, 0.5):
75              keep = lambda q: sig[q["id"]][0] >= c or sig[q["id"]][1] >= b   # noqa:
E731
76              kept = sum(keep(q) for q in ins) / len(ins)
77              if kept < KEEP_TARGET:
78                  continue
79              abst = sum(not keep(q) for q in outs) / len(outs)
80              score = (abst, kept, -c, -b)          # most abstention, then most kept,
then loosest
81              if best is None or score > best[0]:
82                  best = (score, round(float(c), 2), round(float(b), 1))
83      if best is None:
84          raise SystemExit("no gate keeps 95% of in-scope dev questions")
85      return dict(strong_cosine=best[1], min_bm25=best[2])
```

A.11 gateway notfound `serve/gateway.py : 310–330`

```
310                       r"i think it(?:'s| is)|i believe|is (?:that|this|my answer)
(?:right|correct)|"
311                       r"am i (?:right|correct)|did i (?:get|do) (?:it|this)|"
312                       r"where (?:did|have) i (?:go|gone) wrong|check my|"
313                       r"what did i do wrong)\b", re.I)
314 # Fallbacks used only when the classifier is unavailable or returns nothing.
315 _ANALYSIS_HINT = re.compile(r"\b(?:why|compare|comparison|versus|vs\.?|trade-?offs?
|better|"
316                             r"best|worse|choose|choice|which
(?:would|one|scheduler|"
317                             r"algorithm)|design|diagnose|when should|should i|"
318                             r"advantages?|disadvantages?|pros and cons)\b", re.I)
319 _SHELL_HINT = re.compile(r"\b(?:bash|shell|command line|terminal|one-?liner|"
320                          r"(?:which|what) command|grep|awk|sed|chmod|ps aux|kill
-9)\b", re.I)
321
322 OFFTOPIC_REASON = "not an operating-systems question"
323
324 MODE_FOR_KIND = {
325     "formula": "ostutor-solve",
326     "incomplete": "ostutor-solve",     # its prompt asks for the missing input
327     "analysis": "ostutor-analyze",
328     "concept": "ostutor-explain",
329     "shell": "ostutor-shell",
330     "offtopic": "ostutor-explain",     # declined in chat() with OFFTOPIC_REPLY
```

A.12 gateway layers `serve/gateway.py : 393–410`

```
393             yield "data: " + json.dumps(dict(
394                 id=f"ostutor-{how}", object="chat.completion.chunk",
created=int(time.time()),
395                 model=asked, choices=[dict(index=0, delta=dict(content=full[i:i +
400]),
396                                            finish_reason=None)])) + "\n\n"
397         yield "data: [DONE]\n\n"
398     return StreamingResponse(gen(), media_type="text/event-stream")
399
400
401 # --- Verified tutoring ------------------------------------------------------
---
402 # Tutor mode is kept away from the solver's answer so it cannot leak it, and so for
a
403 # while nothing checked what it said: asked about a FIFO attempt of 13 faults, it
walked
404 # the whole trace itself, ignored the three-frame limit, and stated "total page
faults =
405 # 7" (the simulator gives 10). Now the simulator runs privately, the model is told
only
406 # whether the student is right, and the reply is checked before it is shown.
407 NUMERIC_KINDS = ("scheduling", "paging", "disk", "translate")
408 _NUM = re.compile(r"(?<![\w.])-?\d+(?:\.\d+)?(?![\w.])")
409 # The student's own result: "I got 13", "my answer is 4.5", "= 236".
410 # A few words may sit between the verb and the number: "I got an average waiting
time of
```

A.13 mcnemar `ostutor/evaluation/stats.py : 15–30`

```
15  def mcnemar_exact(b: int, c: int) -> float:
16      """Two-sided exact McNemar p-value.
17
18      b = items right under A and wrong under B; c = wrong under A, right under B.
19      Under the null each discordant item is equally likely to fall either way, so
```

```
    min(b, c) is Binomial(b + c, 0.5). Concordant items carry no information.
    Returns 1.0 when there are no discordant items.
    """
    if b < 0 or c < 0:
        raise ValueError("counts must be non-negative")
    n = b + c
    if n == 0:
        return 1.0
    k = min(b, c)
    tail = sum(comb(n, i) for i in range(k + 1)) / 2 ** n
    return min(1.0, 2 * tail)
```

Appendix B Where every number comes from

Numbers	File	Produced by
Dataset composition, pipeline counts, gates	DATASET_CARD.md, reports/manifest.json	python -m ostutor.build
Training and validation loss	outputs/ostutor-lora-v3/ostutor-lora/train_metrics.json	train/kaggle_train.py
Suite, all six configurations, re-graded	reports/g0/20261012-1429/regraded.md	tools/g0_ablation.py, tools/g0_regrade.py
OOD holdout, 600 items	reports/g0/20261012-1429/ood.json	tools/g0_ablation.py
Disk off-by-one, cache determinism	reports/g0/20261012-1429/diagnostics.json	tools/g0_diagnostics.py
Fact-checker probes	reports/g0/20261012-1429/factcheck_v3.json	tools/g0_ablation.py
Forgetting, paired	reports/forgetting_v3_paired.json, reports/g0/historical_significance.md	tools/run_forgetting_eval.py, tools/significance_history.py
RAG corpus and exclusions	data/rag/corpus/manifest.json	tools/import_rag_sources.py, tools/fetch_open_os_sources.py
RAG retrieval and the gate	reports/rag/corpus_eval.md, data/rag/corpus/gate.json	tools/rag_eval.py --calibrate
Embedder comparison	reports/rag/corpus_multilingual-e5-base_eval.md	tools/rag_eval.py --embed-model
RAG end to end, faithfulness	reports/rag/e2e-20261012-2239/ (summary.md, judge_sample_engineer_check.json)	tools/rag_e2e.py
Open-corpus pool analysis, filters, coverage, Figure 1	reports/rag/open_pool_analysis.md, docs/shots/pool_relevance.svg	tools/fetch_rag_pool.py, tools/analyze_rag_pool.py
Corpus comparison by answer recall, ablations	reports/rag/compare_dev.md, compare_test.md, compare_all.md	tools/rag_compare.py
Open-corpus gate	reports/rag/open_gate.json, data/rag/open/gate.json	tools/rag_compare.py --calibrate
Open corpus end to end, short answers, faithfulness	reports/rag/e2e-20261013-1339/, reports/rag/e2e-short-course/, reports/rag/short_e2e-short-course_vs_e2e-20261013-1339.md	tools/rag_e2e.py, tools/rag_e2e_compare.py

Numbers	File	Produced by
Generation speed 26.9 tok/s (wall clock, full stack)	`PROJECT_CONTEXT.md` Section 7	Served stack, 300 tokens
Engineering decisions	`docs/DECISIONS.md` (D-001 … D-009)	—

Reproducing

```
# build the dataset (11 stages, 19 release gates)
python -m ostutor.build

# tests
python tools/test_pipeline.py
python tools/test_router.py
python tools/test_g0.py
python tools/test_rag.py
serve/.venv-webui/Scripts/python serve/test_gateway.py

# evaluation
python tools/g0_ablation.py                  # problem suite, OOD holdout, fact
checker (GPU)
python -m ostutor.rag.ingest data/rag/corpus          # build the retrieval
index (CPU)
python tools/rag_eval.py data/rag/corpus --calibrate  # retrieval + gate (CPU)
python tools/rag_e2e.py data/rag/corpus               # MCQ, explain answers,
faithfulness (GPU)

# serve the tutor with retrieval switched on
$env:OSTUTOR_RAG = "data/rag/corpus"
powershell -File serve\start_tutor.ps1
```